‘**

SECURITY AUDIT

Parallel

Core

FINAL REPORT

Sep 2026

BAILSEC.IO

Disclaimer:

BailSec was engaged by the client to perform a time-boxed technical security assessment of the target identified in this report. The assessment applies only to the scope, version, commit, deployment, materials and information identified in the report. Its findings are point-in-time and not exhaustive. Changes made after the assessment—including remediation changes—were not assessed unless expressly stated otherwise and may affect the applicability of the report's conclusions.

No security assessment can identify every vulnerability or guarantee that a system is secure, error-free or protected against attack or loss. Only the components, assumptions, threat models and risk categories expressly identified as in scope were assessed. The report should be read in full, including its scope, exclusions, unresolved findings and remediation status. The client remains responsible for design, testing, remediation, deployment, operation, monitoring and incident response.

This report is a technical assessment, not a certification, attestation, investment recommendation, or legal, financial, regulatory or tax advice. It does not constitute an endorsement of the client, the assessed system or any related project.

This report was prepared for the client under a separate engagement agreement. If published, its public availability is for transparency and informational purposes only. To the fullest extent permitted by applicable law, publication does not create an auditor-client relationship, duty of care or right of reliance in favor of any third party. Other readers should conduct their own diligence and obtain appropriate professional advice.

Only the complete, unmodified report as issued by BailSec is authoritative. The signed engagement agreement exclusively governs the contractual rights and obligations between BailSec and the client, including warranties, remedies, liability limitations, indemnities and dispute resolution. Nothing in this notice excludes liability that cannot lawfully be excluded.

1. Project Details

Important: Please ensure that the deployed contract matches the source-code of the last commit hash.

Project	Parallel – Core – Audit Report
Website	parallel.best
Language	Solidity
Methods	Manual Analysis
Github repository	https://github.com/parallel-protocol/parallel-parallelizer/tree/0ce59af866f97c3096aa49348d0b709c076ccff4 https://github.com/parallel-protocol/parallel-tokens/tree/63a3416a8a4e597a454cc077266e5651bc28bf56
Resolution 1	https://github.com/parallel-protocol/parallel-parallelizer/tree/95c5ccb90aedfd2590756d6be3362e1e3c0a8ca6 BaseHarvester: https://www.diffchecker.com/4HYiDENL/ GenericHarvester: https://www.diffchecker.com/YmR7DkW6/ parallelizer/Layout.sol: https://diffchecker.com/QjgCki2w parallelizer/Storage.sol: https://diffchecker.com/MnGrrvUh parallelizer/facets/Getters.sol: https://diffchecker.com/HDfjgpnz parallelizer/facets/Redeemer.sol: https://diffchecker.com/6eDEGVWE parallelizer/facets/RewardHandler.sol: https://diffchecker.com/2c3U040i

	parallelizer/facets/SettersGovernor.sol: https://diffchecker.com/VqjR0CyV parallelizer/libraries/LibAuthorization.sol: https://diffchecker.com/GulQ6xar parallelizer/libraries/LibOracle.sol: https://diffchecker.com/oJW2VCMR parallelizer/libraries/LibSetters.sol: https://diffchecker.com/MjQGyUer utils/Constants.sol: https://diffchecker.com/eLeeJk4e utils/Errors.sol: https://diffchecker.com/4ZrYbqx9 Note that the updated commit is now: 518cb9141cf8c51c8d933bae0147e00416d218a6 In-scope changes are only related to the deployment of the oracle.

2. Detection Overview

Severity	Found	Resolved	Partially Resolved	Acknowledged (no change made)	Failed resolution	Open
High						
Medium	11	5		6		
Low	34	7		27		
Informational						
Governance	1			1		
Total	46	12		34		

2.1 Detection Definitions

Severity	Description
High	The problem poses a significant threat to the confidentiality of a considerable number of users' sensitive data. It also has the potential to cause severe damage to the client's reputation or result in substantial financial losses for both the client and the affected users.
Medium	While medium level vulnerabilities may not be easy to exploit, they can still have a major impact on the execution of a smart contract. For instance, they may allow public access to critical functions, which could lead to serious consequences.

Low	Poses a very low-level risk to the project or users. Nevertheless the issue should be fixed immediately
Informational	Effects are small and do not post an immediate danger to the project or users
Governance	Governance privileges which can directly result in a loss of funds or other potential undesired behavior

3. Detection

Parallelizer

Issue_01	Governance Privilege: Proxy Upgrade
Severity	**Governance**
Description	Currently, the contract exposes one or more governance privileges which can result in unexpected operations and potential loss of funds. The whole architecture is upgradeable.
Recommendations	Consider implementing a strong multisig setup for all privileged addresses in the system.
Comments / Resolution	Acknowledged.

BaseHarvester

The BaseHarvester contract is an external contract which is used for rebalancing collateral assets to meet a specific target threshold, mostly related between yield-bearing and non-yield-bearing assets. It is used as an abstract contract which is inherited by the GenericHarvester

Privileged Functions

- setYieldBearingAssetData
- updateLimitExposuresYieldAsset
- setMaxSlippage
- toggleTrusted
- recoverERC20
- setTargetExposure

Issue_02	Rotating harvester asset configuration leaves legacy maximum approvals
Severity	**Low**

Description	Harvesters promote allowances to type(uint256).max and never reduce them.
	
	Governance can later replace the underlying asset associated with a yield-bearing asset, but the setter does not revoke approvals granted for the previous asset relationship.
	
	Deprecated vaults or deposit contracts can therefore retain unlimited authority over old tokens subsequently received by the Harvester.
	
	Root Cause
	
	BaseHarvester._adjustAllowance() effectively grants an unlimited allowance:
	
	if (allowance < amountIn) {
	IERC20(token).safeIncreaseAllowance(
	sender,
	type(uint256).max - allowance
	);
	}
	
	_setYieldBearingAssetData() simply overwrites:
	
	yieldBearingInfo.asset = asset;
	
	It does not inspect or revoke approvals related to the previous asset.

	Approval-producing paths include: • GenericHarvester._swapToTokenOutVault(): underlying token approval to the ERC-4626 vault. • MultiBlockHarvester._rebalance(): underlying-token approval to the configured deposit contract. • Both harvesters: token approvals to the Parallelizer. MultiBlockHarvester.setYieldBearingToDepositAddress() also rotates a spender without revoking allowances held by the old deposit address. Preconditions • A harvest operation has already granted a maximum allowance. • Governance changes the underlying asset or deposit address. • The Harvester later holds the old approved token. • The former spender is compromised, malicious, or no longer trusted. Failure Scenario 1. Yield-bearing asset Y is configured with underlying token A. 2. A vault harvest grants Y an unlimited allowance over A. 3. Governance changes Y's configured underlying to token B. 4. The approval from A to Y remains active. 5. The Harvester later receives A through residual balances, refunds, mistaken transfers, or another workflow. 6. Y can pull that A using the stale allowance. Impact The stale approval does not itself move funds, but it preserves token-spending authority beyond the intended integration lifetime. Any old tokens later held by the Harvester can be drained by the retired spender.
Recommendations	Consider implementing a functionality for governance which allows for resetting allowances manually.

Comments / Resolution	Resolved. On the MultiBlock XEVT path, the contract approves underlying -> depositAddress, while asset rotation clears oldUnderlying -> yieldBearingAsset. The regression test checks that same wrong relationship so it does not cover the actual stale allowance. **It has to be noted that the MultiBlockHarvester is generally not within the audited scope, only the GenericHarvester. Therefore, this issue is marked as fully resolved.**

Issue_03	setMaxSlippage() updates an unused mapping and leaves live harvest limits unchanged
Severity	**Low**

Description	BaseHarvester stores slippage in two different locations:
	
	mapping(address => uint96) public maxTokenSlippage;
	mapping(address => YieldBearingParams) public yieldBearingData;
	
	setMaxSlippage() updates maxTokenSlippage, but every live harvest path reads yieldBearingData[yieldBearingAsset].maxSlippage.
	
	The dedicated setter therefore has no effect on either GenericHarvester or MultiBlockHarvester.
	
	Root Cause
	
	The setter writes:
	
	maxTokenSlippage[yieldBearingAsset] = newMaxSlippage;
	
	However, GenericHarvester.harvest() calculates:
	
	(amount * (1e9 - yieldBearingInfo.maxSlippage)) / 1e9
	
	where yieldBearingInfo was loaded from yieldBearingData.
	
	MultiBlockHarvester also reads the struct's maxSlippage. No scoped contract reads maxTokenSlippage.

	The split is visible in BaseHarvester.sol. Preconditions • The asset was configured through setYieldBearingAssetData(). • Governance later calls the dedicated setMaxSlippage(). • A harvest executes afterward. Failure Scenario 1. An asset is initially configured with 5% maximum slippage. 2. Governance identifies adverse market conditions and calls: setMaxSlippage(asset, 1%); 3. maxTokenSlippage[asset] reports 1%. 4. yieldBearingData[asset].maxSlippage remains 5%. 5. A later GenericHarvester operation accepts 4% slippage even though governance believes the limit is 1%. The reverse also causes operational failures: an attempted loosening does not change the old stricter live limit. Impact A governance mitigation can appear successful on-chain while leaving the actual risk parameter unchanged. Operations may: • execute with more slippage than governance intended; or • continue reverting after governance believes it has relaxed the limit.
Recommendations	Consider handling the storage update for the correct parameter.
Comments / Resolution	Fixed.

GenericHarvester

The GenericHarvester contract is a permissionless rebalancing contract that inherits from BaseHarvester and adjusts the Parallelizer's exposure between yield-bearing assets and their underlying collateral assets. It uses zero-fee tokenP flash loans to execute the rebalance and supports conversions through either an external swap router or an ERC-4626 vault, while user-funded budgets cover any tokenP shortfall produced by the operation.

Privileged Functions

- setTokenTransferAddress
- setSwapRouter

Issue_04	Anyone can time the irreversible oracle ratchet
Severity	**Medium**

Description	For collaterals using OracleReadType.MAX, LibOracle.updateOracle() permanently raises the stored target to the current raw spot price. The update has no maximum increase, deviation bound, cooldown, or time-weighting.
	
	Although SettersGovernor.updateOracle() requires a trusted seller, a trusted GenericHarvester calls it from the permissionless harvest() function. Consequently, any address can choose the block in which the target is ratcheted.
	
	A transient upward oracle print can therefore become the permanent target. Once spot returns to normal, readBurn() interprets the difference as a depeg and applies a lasting burn haircut. Because getBurnOracle() takes the minimum ratio across every collateral, one affected collateral discounts burns for all collateral assets.
	
	Root Cause
	
	uint256 oracleValue = read(oracleType, BASE_18, oracleData);
	
	uint256 maxValue = abi.decode(targetData, (uint256));
	
	if (oracleValue > maxValue) {
	
	ts.collaterals[collateral].oracleConfig = abi.encode(
	
	oracleType,

```
        targetType,
        oracleData,
        abi.encode(oracleValue),
        hyperparameters
    );
}
```

The raw value is stored without validating whether the increase is economically meaningful or temporary.

Failure Scenario

1. A MAX collateral normally trades at 1.00.
2. Its oracle temporarily reports 1.10.
3. Any caller invokes GenericHarvester.harvest().
4. The trusted Harvester calls updateOracle(), permanently storing 1.10.
5. Spot returns to 1.00.
6. readBurn() now reports a ratio of approximately 1 / 1.10.
7. getBurnOracle() propagates this ratio to burns of every collateral.

Impact

All burn users receive less collateral than before, even when burning an unaffected asset. A sufficiently inflated target can reduce exact-input burn output to zero and make exact-output burns prohibitively expensive or impossible.

The state persists until governance replaces the entire oracle configuration.

Furthermore, this mechanism defeats the purpose of a target clamp of preventing an upward SPOT price movement to be used as value.

Recommendations	Consider making the harvest permissioned.
Comments / Resolution	Resolved. The change additionally introduces a max increment of 5% for the oracle, but the price is initialized as 0 in the deployment script. So oracles can never be updated, since any non-zero update will be above 5%. **This is not an issue for upgrades as the oracle price is already > 0 but for new protocol deployment, the script should be adjusted. Furthermore, any price increase above 5% will permanently DoS oracle updates until governance re-sets it.**

Issue_05	Permissionless genericHarvester execution bypasses final-recipient collateral whitelisting
Severity	**Medium**
Description	GenericHarvester.harvest() is permissionless and accepts caller-selected router calldata. During a decrease-exposure rebalance, GenericHarvester: 1. flash-borrows USDp; 2. burns the USDp for the configured yield-bearing collateral; 3. receives that collateral itself; 4. swaps it for the associated base collateral; 5. mints USDp with the base collateral; and 6. repays the flash loan. When the Parallelizer releases the restricted collateral, Swapper._swap() checks whether the supplied recipient is whitelisted. GenericHarvester passes because the recipient is address(this). The whitelist does not follow the collateral after it reaches GenericHarvester. Root Cause GenericHarvester is both: • an authorized recipient of restricted collateral; and • a permissionless executor of caller-supplied router calldata. RouterSwapper approves the configured Odos transfer address and forwards the calldata without binding the route's input receiver, output receiver, or executor. The Odos payload can specify: • an input receiver for the restricted collateral;

	• an executor called through executePath(); and • an output receiver for the resulting token. Odos measures output by checking how much tokenOut arrives in the router during executePath(). It does not require that this output was economically produced from the supplied tokenIn. An externally funded executor can therefore provide the required output while the restricted input is delivered elsewhere. Preconditions • The relevant collateral is recipient-whitelisted. • GenericHarvester itself is whitelisted. • The collateral is configured as a harvestable yield-bearing asset. • Current exposure requires a decrease of that yield-bearing asset. • harvest() calculates a nonzero rebalance amount. • The caller selects the router-based swap mode. • Caller-controlled calldata directs the restricted input to an unwhitelisted recipient. • The selected executor supplies sufficient tokenOut to Odos. • Odos forwards the output to GenericHarvester. • The resulting USDp mint covers the flash-loan principal, with any permitted shortfall covered by the caller's budget. Failure Scenario GenericHarvester flash-borrows USDp and burns it for a whitelist-restricted yield-bearing collateral. Swapper checks GenericHarvester as the immediate recipient and releases the collateral. The caller-supplied Odos payload directs the restricted collateral to an unwhitelisted address. During executePath(), an executor funded with the required base collateral transfers that base collateral into the Odos router. Odos records the balance increase as swap output and transfers it to

	GenericHarvester. GenericHarvester receives the expected output token, mints sufficient USDp and repays the flash loan. The entire transaction completes while the unwhitelisted recipient retains the restricted collateral. Impact An unwhitelisted actor can acquire recipient-restricted collateral by economically exchanging unrestricted collateral for it through GenericHarvester. The Parallelizer can remain fully solvent, and the flash loan can be repaid. The broken invariant is the collateral recipient restriction, not necessarily reserve accounting. If the whitelist represents a compliance, sanctions, transferability, or asset-security boundary, GenericHarvester becomes a permissionless laundering intermediary around that boundary.
Recommendations	Consider implementing a role-based access system on the harvester mechanism.
Comments / Resolution	Fixed.

Issue_06	Non-zero flashloan fee will deactivate harvester capabilities
Severity	**Low**
Description	GenericHarvester relies on FlashParallelToken to temporarily mint USDp during an atomic collateral rebalance. FlashParallelToken explicitly supports configurable flash-loan fees. Governance can update the fee rate through setFlashLoanParameters(). The lender calculates the fee and passes it to GenericHarvester.onFlashLoan(). Root Cause GenericHarvester.onFlashLoan() accepts the callback only when the fee is zero. A callback from the correct immutable lender with the correct initiator is still rejected whenever the lender reports a nonzero fee. The Harvester does not query the fee before requesting the loan and has no alternative rebalance path that avoids the flash lender. Preconditions • Governance configures a nonzero USDp flash-loan fee. • The harvest amount is large enough for the calculated fee to be nonzero after rounding. • A caller invokes GenericHarvester.harvest(). • The rebalance requires the configured flash lender. A nonzero rate may still produce a zero fee for dust amounts, but meaningful harvest amounts will be affected. Failure Scenario Governance enables a small USDp flash-loan fee through the functionality provided by FlashParallelToken. A user calls harvest() for an economically meaningful rebalance.

	The flash lender temporarily mints USDp and calls onFlashLoan() with the calculated fee. GenericHarvester rejects the callback because the fee is nonzero. The transaction reverts before any rebalance can complete. All subsequent meaningful harvests encounter the same failure. Impact GenericHarvester can no longer adjust the Parallelizer's yield-bearing collateral exposure. The protocol may remain outside its configured target exposure and may be unable to restore sufficient liquid collateral during stressed conditions. The failure is atomic and does not directly corrupt accounting or lose assets.
Recommendations	Consider ensuring that the fee remains zero.
Comments / Resolution	Fixed.

Issue_07	Residual swap inputs and standing router approvals can be extracted through a later permissionless harvest
Severity	**Low**

Description	GenericHarvester assumes the router consumes the full declared input amount, but verifies only the increase in the expected output-token balance.
	
	The imported router helper:
	
	• approves tokenTransferAddress for the declared amount;
	• calls the configured router with caller-controlled calldata;
	• does not measure the actual input spent; and
	• does not clear the remaining allowance.
	
	If a route partially consumes its input, both the residual collateral and the unused allowance remain on the Harvester. Because harvest() is permissionless and accepts arbitrary router calldata, a later caller may use the standing allowance to extract or redirect the residual through the configured router.
	
	The ultimate exploit depends on the router supporting the necessary recipient, multicall, multi-output, or partial-fill semantics.
	
	Root Cause
	
	_swapToTokenOutSwap() snapshots only tokenOut:
	
	uint256 balance =
	  IERC20(tokenOut).balanceOf(address(this));
	
	_swap(tokens, callDatas, amounts);
	
	return
	  IERC20(tokenOut).balanceOf(address(this)) - balance;
	
	It never checks:
	
	tokenInBalanceBefore - tokenInBalanceAfter == amount;
	
	The imported RouterSwapper.sol merely ensures the allowance is large enough and calls the router:
	
	_approveTokenIfNeeded(tokens[i], tokenTransferAddress, amounts[i]);

	_performRouterSwap(callDatas[i]); A partial fill leaves the unspent portion both: • in the Harvester's balance; and • approved to tokenTransferAddress. GenericHarvester.harvest() is public and passes caller-controlled extraData into this flow. See GenericHarvester.sol. Preconditions • The configured router or transfer proxy supports partial input consumption. • The initial harvest still produces at least minAmountOut; otherwise it reverts. • A nonzero residual input and residual allowance remain. • A later rebalance opportunity exists because harvest() reverts when the computed amount is zero. • The configured router permits calldata that can consume the historical residual and direct value to the attacker or otherwise subsidize the attacker's operation. • The later harvest still satisfies its own output and flash-loan repayment requirements. A payload that simply steals the residue while returning zero expected output will revert the entire transaction. Failure Scenario 1. A harvest obtains 100 units of tokenIn from the Parallelizer. 2. The router spends only 95 units but produces sufficient tokenOut to satisfy the live slippage requirement. 3. Five tokenIn remain in the Harvester. 4. The transfer proxy retains allowance for those five units. 5. During a later nonzero rebalance, an attacker calls permissionless harvest() with crafted router calldata. 6. The calldata consumes the historical five-unit allowance and routes that value to an attacker-controlled output or recipient.

	7. The payload simultaneously produces enough of the current expected output token for the new harvest to complete. Impact Protocol-origin collateral left by partial fills can become extractable through a later permissionless call. Even where extraction is unsupported by the deployed router, the residual becomes stranded outside the Parallelizer's normal collateral accounting until governance invokes recoverERC20(). The loss is bounded by accumulated residual balances and remaining approvals. A High classification requires demonstrating that the actual deployed router can: • partially consume inputs; • use historical approvals for a different leg; and • direct some resulting value away from the Harvester while satisfying the current harvest.
Recommendations	Consider acknowledging this issue.
Comments / Resolution	Fixed.

Issue_08	Positive harvester settlement is not credited to the initiating caller's budget
Severity	**Low**
Description	GenericHarvester charges an unsuccessful rebalance against the

initiating caller's TokenP budget. It does not perform the inverse accounting when the same operation produces more TokenP than required to repay the flash loan.

The excess TokenP remains in the Harvester's balance without being attributed to any caller in the budget mapping. The initiator therefore cannot withdraw it through removeBudget().

Root Cause

Users deposit TokenP into the Harvester and receive an equal budget balance:

```
function addBudget(uint256 amount, address receiver) public {
  budget[receiver] += amount;
  IERC20(tokenP).safeTransferFrom(
    msg.sender,
    address(this),
    amount
  );
}
```

When initiating a rebalance, the caller's address is encoded in the flash-loan callback data:

```
abi.encode(
  msg.sender,
  increase,
  yieldBearingAsset,
  asset,
  minAmountOut,
  swapType,
  extraData
)
```

After completing the round trip, onFlashLoan() calculates the received TokenP:

```
uint256 amountStableOut =
```

```
 parallelizer.swapExactInput(
  amountOut,
  minAmountOut,
  tokenOut,
  address(tokenP),
  address(this),
  block.timestamp
 );
```

A shortfall is charged to the caller:

```
if (amount > amountStableOut) {
 budget[sender] -= amount - amountStableOut;
}
```

There is no positive settlement branch when:

amountStableOut > amount

The flash lender subsequently retrieves only amount, because the callback requires fee == 0. The remaining TokenP stays in GenericHarvester.

Preconditions

- A caller successfully initiates harvest().
- The flash-loan fee is zero, as required by the callback.
- The burn-conversion-mint route produces more TokenP than the borrowed amount.
- The positive result is not consumed by another operation before settlement.

A positive result may arise from favorable fee curves, favorable external execution, yield accrued in a converted asset, or rounding across the route.

Failure Scenario

1. The Harvester flash-borrows 1,000 USDp.
2. It burns the USDp, converts the collateral and mints 1,010 USDp.

	3. The flash lender retrieves and burns the 1,000 USDp principal. 4. 10 USDp remains in GenericHarvester. 5. budget[sender] is not increased. 6. The caller cannot withdraw the 10 USDp because removeBudget() subtracts from the unchanged budget mapping. Impact Rebalance settlement is asymmetric: • negative performance is charged to the initiating caller; • positive performance becomes an unallocated contract balance. This can discourage permissionless callers from financing or executing rebalances because they bear the downside without receiving the corresponding upside. It also causes the Harvester's actual TokenP balance to exceed the aggregate amount represented by user budgets. The excess can later be moved only through restricted recovery functionality unless another explicit accounting path is introduced. The issue is distinct from incomplete router consumption. That issue leaves residual tokenIn after a partial external fill; this issue occurs after a completed route and concerns the positive TokenP settlement delta.
Recommendations	Consider recovering any idle funds via governance.
Comments / Resolution	Fixed.

AccessManagedModifiers

The AccessManagedModifiers contract provides shared access-control and reentrancy-protection modifiers for the Parallelizer's Diamond facets. It validates restricted calls through the Diamond's configured authority and stores the reentrancy status in the shared Parallelizer storage to prevent direct and indirect reentrant execution across facets.

Privileged Functions

- none

No issues found.

Getters

The Getters contract is a read-only Diamond facet that exposes the Parallelizer's configuration, accounting, collateral, oracle, access-control, and operational state. It provides information including supported collateral assets, issuance amounts, collateral ratios, fee and redemption curves, manager configurations, pause states, whitelist status, surplus parameters, and protocol balances.

Privileged Functions

- none

No issues found.

Redeemer

The Redeemer contract is a Diamond facet that allows tokenP holders to burn their tokens in exchange for a proportional share of the Parallelizer's collateral reserves. It supports direct and EIP-3009-authorized redemptions, optional forfeiture of selected output tokens, managed

collateral withdrawals, whitelist enforcement, undercollateralization penalties, and updates to the issuance normalizer

Privileged Functions

- updateNormalizer

Issue_09	Redemption minima are indexed against a mutable output-token list
Severity	**Medium**
Description	Redemption calldata commits only to an array of minimum amounts. It

	does not identify which output token each minimum protects. The actual output-token list is rebuilt at execution time from the mutable collateralList. Governance can revoke collateral using swap-and-pop and add other collateral, changing both token identities and ordering while preserving the output-array length. The issue exists in both ordinary and authorized redemptions. EIP-3009 makes it stronger because a signed authorization can remain executable by a relayer throughout its validity window. Root Cause LibAuthorization.computeRedeemNonce() binds: • the owner; • redemption amount; • receiver; • deadline; • the hash of minAmountOuts; • the hash of forfeitTokens; and • the user salt. It does not bind the token array returned by _quoteRedemptionCurve(). At execution: 1. _quoteRedemptionCurve() builds tokens from the current collateralList. 2. _redeem() checks only that the current amounts array has the same length as minAmountOuts. 3. _distributeRedemption() compares each current output amount against the minimum at the same numeric index. There is no validation that index zero still represents the token the user expected at signing or submission time. Preconditions • The collateral configuration changes between quote/signing and

execution.

- The final output list has the same length as the signed or submitted minimum array.
- At least one token identity or position changes.
- The mismatched raw minimum values still pass, such as when minima are zero, conservative, or the affected tokens use compatible decimals.
- For authorized redemption, a relayer retains a still-valid signature and chooses when to execute it.
- For ordinary redemption, reconfiguration must occur before transaction inclusion.

Failure Scenario

1. A user obtains a redemption quote for tokens A, B, and C and signs minima in that order.
2. Before execution, governance revokes B. Swap-and-pop moves C into B's position.
3. Governance adds D, restoring the list to three tokens.
4. The live output order becomes A, C, and D.
5. The authorization remains valid because the signed minima are unchanged and no token list was included in the derived nonce.
6. If the raw minima pass, the user burns USDp and receives A, C, and D rather than the expected A, B, and C.

Impact

A user can receive an unexpected collateral composition despite supplying or signing slippage constraints. Those constraints can protect raw quantities while failing to protect token identity or economic value.

A malicious relayer cannot independently reconfigure collateral, but can opportunistically execute an existing authorization after a legitimate governance migration.

Source Verification

- LibAuthorization.sol, computeRedeemNonce(): no token-list hash

	is included. • LibGetters.sol, getCollateralRatio(): output tokens are generated from the current collateralList. • Redeemer.sol, _redeem() and _distributeRedemption(): only array length and per-index amounts are checked. • LibSetters.revokeCollateral(): removal uses swap-and-pop. • LibSetters.addCollateral(): newly added collateral is appended.
Recommendations	Consider adding an expectedTokenHash into the nonce and as argument into the redemption functionalities. This should then compared against the actual redemption tokens (via hash). It has to be noted that this will invalidate all previous signatures and will not be compatible with previous ABI.
Comments / Resolution	Fixed.

Issue_10	Recoverable oracle-overvalued collateral donations inflate the redemption coefficient and extract excess healthy reserves
Severity	**Medium**
Description	LibGetters.getCollateralRatio() includes the Parallelizer's raw ERC-20 collateral balances, regardless of whether those balances entered

	through a mint. A user can directly donate an oracle-overvalued collateral to the Parallelizer. The donation increases the collateral-ratio numerator without increasing tracked issuance. During undercollateralization, this moves the redemption curve to a more favorable coefficient. The attacker can then redeem TokenP for the whole basket, extracting additional healthy reserves. Because the donated collateral is included in the redemption output, it is partially recoverable and can be transferred back to the Parallelizer before another redemption. Forfeiting the donated asset produces a similar state transition: it remains in reserves and is counted again after liabilities decrease. Root Cause For unmanaged collateral, LibGetters reads: collateralBalance = IERC20(collateralList[i]).balanceOf(address(this)); It then includes the balance in: totalCollateralization There is no distinction between: • collateral received during an accounted mint; • collateral transferred as a donation; • collateral previously forfeited during redemption; and • collateral recovered and re-donated by a redeemer. Tracked issuance remains: normalizedStables * normalizer / BASE_27 The redemption coefficient is selected from: collateralRatio = total oracle-valued collateral / tracked issuance

Under the undercollateralized branch, healthy collateral output is approximately:

amountBurnt
× healthyBalance / stablecoinsIssued
× penaltyFactor(collateralRatio)

An overvalued donation can cheaply increase penaltyFactor for every collateral.

Preconditions

- The system is undercollateralized or close to a material redemption-curve breakpoint.
- An accepted collateral's redemption oracle materially exceeds its realizable market value.
- Direct ERC-20 transfers to the Parallelizer are possible.
- The redemption curve increases with collateral ratio.
- The attacker owns or can acquire a material TokenP position.
- Redemption is active.
- The attacker's donation moves the reported ratio into a more favorable curve region.
- The additional healthy collateral extracted exceeds the unrecovered market cost of the donation.

Failure Scenario

Assume:

Tracked TokenP liabilities: 100
Healthy collateral H: 80
Existing overvalued collateral X: 0
Reported collateral ratio: 80%

Penalty at 80% CR: 50%
Penalty at 90% CR: 90%

X oracle price: $1

	X market price: $0.10 The attacker buys and donates 10 X: Market cost: $1 Oracle value added: $10 New reported CR: 90% The attacker redeems 50 TokenP. The quote now returns: Healthy H: 50 × 80 / 100 × 90% = 36 H Donated X: 50 × 10 / 100 × 90% = 4.5 X Without the donation, the healthy output at the original 50% coefficient would have been: 50 × 80 / 100 × 50% = 20 H The donation therefore increases the first redemption's healthy output by 16 H while returning 4.5 of the donated X. Recycling Escalation After the first redemption: Remaining liabilities: 50 Remaining H: 44 Remaining X: 5.5 The attacker can transfer the received 4.5 X back to the Parallelizer: X balance restored to: 10 Reported collateral: 54 Reported CR: 108%

The same initial donation is now counted again after liabilities have fallen from 100 to 50.

The attacker can redeem again under the overcollateralized branch, extract most of the remaining healthy collateral, and receive most of X back during the final redemption. The capital used to manipulate the coefficient is therefore largely recoverable.

Forfeiture Escalation

Instead of receiving and manually re-donating X, the attacker can include X in forfeitTokens.

The redemption then:

- burns the full TokenP amount;
- transfers healthy collateral;
- leaves X inside the Parallelizer; and
- immediately exposes the retained X to the next collateral-ratio calculation.

This is economically similar to recycling, but it avoids the transfer-out and transfer-back step. It is most attractive when X has little market value but substantial oracle value.

Impact

An attacker can bypass intended undercollateralization penalties and extract excess healthy reserves using comparatively cheap, oracle-overvalued collateral.

The process:

- drains liquid and correctly valued collateral;
- leaves remaining holders with a higher concentration of impaired collateral;
- makes the reported collateral ratio diverge further from realizable backing;
- can be repeated by recycling the same donated collateral; and

	• can move later redemptions into the unpenalized overcollateralized branch.
Recommendations	The developer has been notified during the audit and the fix was to smoothen out the curve parameters to prevent profitability.
Comments / Resolution	Resolved. The curve has been modified to make the vector less profitable, but drastic price differences can still lead to a profitable scenario. Combining the test's 95% collateralization, 90% holder and 2% donation with the report's 10% oracle acquisition price still gives about 0.1709% of supply in incremental net profit before gas and rounding.

Issue_11	Address-based forfeiture can force users to forfeit an otherwise claimable standalone collateral
Severity	**Medium**
Description	Redemption forfeiture is identified only by token address. It does not distinguish between different occurrences of the same token in the

	flattened redemption output.
	
	A managed collateral may expose multiple sub-collaterals. The configuration validation only requires the first sub-collateral to equal the parent collateral; it does not prevent another sub-collateral from also being registered independently as a regular collateral.
	
	If managed collateral A contains token B as a sub-collateral while B is also a standalone collateral, the redemption output can contain B twice:
	
	[A, B, B, C]
	
	If the receiver is not whitelisted for managed collateral A, the user must forfeit the first B because it inherits A's whitelist requirement. Since forfeiture is checked globally by token address, the second, independently claimable B is also forfeited.
	
	Root Cause
	
	LibSetters.setCollateralManager() validates only the first configured sub-collateral:
	
	if (address(managerData.subCollaterals[0]) != collateral)
	revert InvalidParams();
	
	It does not prevent a sub-collateral from also appearing in collateralList.
	
	LibGetters.getCollateralRatio() then creates a flattened redemption array. Managed collateral outputs are appended from managerData.subCollaterals:
	
	tokens[countCollat + k] =
	address(collateral.managerData.subCollaterals[k]);
	
	Standalone collaterals are appended separately:
	
	tokens[countCollat] = collateralList[i];
	
	Consequently, the same token address can appear multiple times under different parent-collateral contexts.

During distribution, Redeemer._distributeRedemption() applies the parent collateral's whitelist to each sub-collateral:

```
Collateral storage collatInfo =
 ts.collaterals[collateralListMem[indexCollateral]];

if (
 collatInfo.onlyWhitelisted > 0
  && !LibWhitelist.checkWhitelist(
   collatInfo.whitelistData,
   redemption.to
  )
) {
 revert NotWhitelisted();
}
```

The forfeiture check, however, does not incorporate indexCollateral or the parent collateral:

```
LibHelpers.checkList(
 redemption.tokens[i],
 redemption.forfeitTokens
) < 0
```

It therefore treats all occurrences of a token as equivalent.

Preconditions

- A managed collateral A has token B as one of its sub-collaterals.
- Token B is also registered as an independent collateral.
- Managed collateral A requires its redemption receiver to be whitelisted.
- The intended receiver is not whitelisted for A.
- Both occurrences of B have a nonzero redemption allocation.
- The user wants to redeem without using a different whitelisted receiver.

Failure Scenario

Assume:

collateralList = [A, B, C]
A.subCollaterals = [A, B]

The redemption output becomes:

[A, B, B, C]

The loop processes the outputs as follows:

1. A is processed under managed collateral A.
2. The first B is also processed under A, because indexCollateral does not advance until every sub-collateral of A has been processed.
3. The second B is processed as standalone collateral B.
4. C is processed as standalone collateral C.

Because the receiver is not whitelisted for A, the user must include both A and the first B in forfeitTokens; otherwise the redemption reverts.

The user therefore submits:

forfeitTokens = [A, B]

When the loop reaches standalone B, the address-based lookup finds B in forfeitTokens and skips that transfer as well, even though standalone B does not require forfeiture.

Impact

Users may be forced to surrender collateral that they are otherwise eligible to receive.

The loss can include the entire standalone redemption allocation of the duplicated token. The forfeited collateral remains in protocol custody or under the relevant manager while the user's TokenP has already been burned.

The issue affects both ordinary and authorization-based redemptions

	because both routes eventually use the same flattened output and address-based forfeiture logic.
Recommendations	Since no manager is used and this issue has already been previously acknowledged, we recommend acknowledging it.
Comments / Resolution	Acknowledged.

Issue_12	Negative mint rebates bypass the direct-burn fee-safety invariant through basket redemption
Severity	**Medium**
Description	LibSetters.checkFees() ensures that a negative mint fee cannot be combined with any direct-burn fee to produce a profitable

	mint-and-burn cycle. It does not compare negative mint rebates against the redemption coefficient. A user can mint more TokenP than the value deposited and redeem those TokenP through Redeemer, bypassing every direct-burn fee used by the safety check. The route is profitable whenever the effective redemption coefficient is larger than the mint fee's post-rebate denominator. Root Cause When installing a negative mint fee, checkFees() only validates: burnFees[0] + negativeMintFee >= 0 for every direct-burn collateral. It does not inspect: xRedemptionCurve yRedemptionCurve Similarly, updating the redemption curve does not revalidate existing negative mint fees. For a negative mint fee with absolute rebate r, depositing collateral worth V produces approximately: TokenP minted = V / (1 - r) When the protocol is overcollateralized, redemption returns aggregate oracle value approximately equal to: redemption value = TokenP redeemed × yFinal where yFinal is the last redemption-curve coefficient. The cycle is profitable when: yFinal > 1 - r

Equivalently:

mint rebate > redemption haircut
Preconditions

- A collateral has a negative mint fee.
- The negative mint fee is larger than the applicable redemption haircut.
- Redemption remains active.
- The system is sufficiently collateralized that the resulting redemption coefficient remains favorable.
- There is existing issuance, because the user cannot redeem the complete global tracked issuance.
- The user's mint and redemption remain within favorable exposure and curve regions.
- Oracle prices do not offset the rebate.

Negative fees require governor participation when initially configured, but exploitation thereafter is permissionless.

Failure Scenario

Assume:

Initial tracked issuance: 1,000,000 TokenP
Initial collateral value: $1,010,000
Mint fee for collateral A: -1%
Final redemption coefficient: 99.5%

The user deposits collateral A worth $100:

TokenP minted
= 100 / (1 - 1%)
≈ 101.0101 TokenP

The direct-burn invariant may require every burn route to charge at least 1%, preventing profitable direct burning.

Instead, the user redeems through Redeemer. Because the system

	remains overcollateralized, the aggregate collateral value returned is approximately: 101.0101 × 99.5% ≈ $100.505 The user's gross gain is approximately: $100.505 - $100 = $0.505 The mint increases tracked issuance by approximately 101.0101. Redemption subsequently decreases tracked issuance by the same amount, so global issuance returns approximately to its starting value while collateral reserves lose the user's profit. Undercollateralized Condition If the mint moves the system below 100% collateralization, aggregate redemption value becomes approximately: TokenP redeemed × collateralRatio × penaltyFactor The route remains profitable when: collateralRatio × penaltyFactor > 1 - rebate A large existing system means a small user mint may only marginally change the collateral ratio, preserving profitability. Impact Users can repeatedly convert collateral A into a larger oracle value of the entire collateral basket. This drains reserves while bypassing the exact direct-burn fees that checkFees() relies on to establish safety. The drain continues until: • the protocol becomes sufficiently undercollateralized; • redemption penalties increase; • the negative mint fee moves into a less favorable exposure

	segment; • A reaches its mint cap; or • redemption is paused.
Recommendations	Consider not setting negative mint fees.
Comments / Resolution	Acknowledged.

Issue_13	Uncapped premium collateral can suppress redemption penalties and accelerate healthy reserve withdrawals
Severity	**Low**
Description	LibOracle.readMint() caps a non-external oracle value at the collateral's target price. A collateral trading above target therefore cannot produce

	additional USDp during minting. LibOracle.readRedemption() does not apply the same cap. LibGetters.getCollateralRatio() values the entire collateral balance using this uncapped redemption price. The resulting collateral ratio determines which redemption-curve segment applies and how much of each reserve asset the user receives. Root Cause The system deliberately rejects a collateral premium when determining how much USDp can be issued, but accepts the same premium when determining whether USDp holders should receive an undercollateralization haircut. This creates inconsistent treatment of the same collateral value across issuance and redemption. The inconsistency is material when the uncapped premium moves the reported collateral ratio across a redemption-curve boundary. In particular, it can move the protocol from an undercollateralized segment into the unpenalized or minimally penalized segment. Preconditions • A supported collateral is reported above its target price. • The target-capped basket value would place the protocol in a less favorable redemption segment. • The uncapped spot value places it in a more favorable segment. • The reported premium is not fully realizable or is likely to disappear. • A user holds USDp and can redeem before the premium disappears. • Sufficient healthy collateral remains available for distribution. Failure Scenario Assume the protocol has 100 million USDp of tracked issuance.

	The basket contains 80 million dollars of healthy collateral and 10 million units of another collateral. That collateral has a one-dollar target but is temporarily reported at two dollars. Using the uncapped redemption price, the basket is reported as worth 100 million dollars and the redemption enters the fully collateralized branch. A user redeems 50 million USDp and receives approximately half of every reserve asset, subject only to the terminal redemption fee. If the premium collateral can actually be realized for only one dollar per unit, the basket was economically worth only 90 million dollars. The user has therefore avoided the stronger haircut that should have applied at a 90% collateral ratio and has removed substantially more healthy collateral. Impact Early redeemers can withdraw more healthy reserves than intended by the undercollateralization curve. The loss is transferred to later USDp holders because the premium collateral remains in the residual basket or subsequently falls back toward its target price. The impact is conditional. If the premium collateral is genuinely worth the reported spot price and can be liquidated at that price, no accounting loss follows from the uncapped valuation.
Recommendations	This is not considered as a severe issue per-se. It is however an inconsistency in the cap application which can be considered as a design decision. We recommend documenting this behavior as design choice in the natspec. Using an uncapped spot price for redemption is not independently unsafe. If the reported premium is genuine and realizable, the protocol is correctly recognizing valuable collateral.

	The material issue arises when the spot premium is temporary, stale, manipulable, or cannot be realized by the protocol. In that state, the premium can make the protocol appear sufficiently collateralized to apply a more favorable redemption factor. This is usually a theoretical likelihood only.
Comments / Resolution	Acknowledged.

Issue_14	Near-empty renormalization can drop issuance dust and skew future allocation updates
Severity	**Low**
Description	When the normalizer becomes very small, _updateNormalizer() converts every collateral's normalized issuance into an absolute integer using floor division.

Each collateral can lose less than one TokenP base unit during this conversion. A collateral whose remaining allocation is below one base unit is reset to zero.

The immediate discrepancy is only dust. However, a later large positive normalizer update uses these rounded integer allocations as its weights. Zeroed collateral receives none of the increase, while a tiny intervening mint can materially redefine the allocation weights.

Root Cause

Renormalization performs independent floor division for each collateral:

```
newCollateralNormalizedStable =
  collateral.normalizedStables
    * newNormalizerValue
    / BASE_27;
```

It then defines the new global value as the sum of those floored values:

```
newNormalizedStables += newCollateralNormalizedStable;
ts.normalizedStables = newNormalizedStables;
newNormalizerValue = BASE_27;
```

No remainder is preserved or redistributed.

Exact Failure Scenario

Assume:

```
normalizer          = 1e27
global normalized   = N = 1e27
X allocation        = 49% of N
Y allocation        = 51% of N
```

A holder redeems:

N - 2 base units

This is permitted because the redemption is strictly smaller than

	stablecoinsIssued. The new normalizer becomes: newNormalizer = 2 Renormalization calculates: X: floor(0.49 × 2) = 0 Y: floor(0.51 × 2) = 1 The system therefore records one remaining issuance unit even though two TokenP base units remain after the burn. X's allocation is reset to zero, and Y becomes the only nonzero issuance weight. If a large positive updateNormalizer() later occurs, its entire increase is attributed to Y. Alternatively, a small mint into X before that update can redefine the weights at negligible cost relative to the forthcoming increase. Preconditions • Almost the entire tracked TokenP supply is redeemed. • The resulting normalizer reaches the renormalization threshold. • At least one collateral allocation rounds to zero while another survives. • A later positive normalizer update is large relative to the dust issuance. • For sequencing manipulation, an attacker can mint before a predictable trusted update. Impact Immediate impact is strictly dust-bounded: less than one TokenP base unit can be lost per supported primary collateral. The larger effect is accounting-distribution skew: • getIssuedByCollateral() can assign future global issuance to

	different collateral than intended. • Cap consumption and exposure-dependent fees follow the rounded weights. • Harvester targets and collateral-specific surplus calculations may be affected. This does not reset collateral reserves and does not directly enable a reserve drain.
Recommendations	This issue can be acknowledged because no updateNormalizer call will ever be executed.
Comments / Resolution	Acknowledged.

RewardHandler

The RewardHandler contract is a Diamond facet used by governance or trusted sellers to convert non-collateral reward tokens into supported collateral through the Odos router. It verifies that the operation does not reduce the Parallelizer's tokenP or collateral balances and automatically transfers and invests newly obtained collateral when that collateral is managed by an external strategy.

Privileged Functions

- sellRewards

Issue_15	RewardHandler calls the ethereum odos router on non-ethereum deployments
Severity	**Low**
Description	Constants.sol defines ODOS_ROUTER as: 0xCf5540fFFCdC3d510B18bFcA6d2b9987b0772559 RewardHandler.sellRewards() performs a direct low-level call to this

address.

Unlike GenericHarvester, RewardHandler has no configurable swapRouter storage variable and receives no router address during deployment.

The official Odos deployment list identifies the hardcoded address as the Ethereum router. It lists different addresses for Base, Avalanche, Sonic, and other chains.

Root Cause

A chain-specific integration address is compiled into a facet that is deployed across multiple chains.

The RewardHandler deployment process cannot substitute the correct router address for each chain because the dependency is a Solidity constant.

Preconditions

- The Parallelizer deploys the RewardHandler facet outside Ethereum.
- Odos is not deployed at the Ethereum address on that chain.
- Governance grants reward-token allowance to the address expected by the RewardHandler flow.
- A trusted seller calls sellRewards().

These conditions apply to the repository's non-Ethereum RewardHandler deployments unless a compatible router has independently been deployed at the Ethereum address.

Failure Scenario

A trusted seller calls sellRewards() on Base.

RewardHandler calls the hardcoded Ethereum address instead of the Base Odos router.

If the address has no bytecode, the low-level call can return success without executing a swap. RewardHandler then observes that no

	collateral balance increased and reverts with InvalidSwap. If unrelated code exists at the address, the trusted payload is sent to an unintended contract. The post-swap balance checks reduce the direct reserve-loss surface but do not make the reward conversion valid. Impact The normal RewardHandler conversion path is unusable on affected chains. Reward tokens remain in the Parallelizer until governance manually recovers and converts them or replaces the facet. This delays surplus realization and can prevent the expected flow of reward income into protocol collateral. The failure does not silently succeed: the missing collateral increase normally causes the transaction to revert atomically.
Recommendations	Consider defining the correct router address for each chain.
Comments / Resolution	Fixed.

Issue_16	Multi-output odos sales leave earlier managed collateral outside manager
Severity	**Low**

Description	RewardHandler.sellRewards() permits an Odos payload to increase more than one supported collateral balance. During the post-swap balance loop, each increase overwrites the single collateral and amountOut variables:
	
	collateral = list[i];
	amountOut = newBalance - balances[i];
	
	After the loop, manager processing is performed only once for the final

increased collateral:

Collateral storage collatInfo = ts.collaterals[collateral];

```
if (collatInfo.isManaged > 0) {
   IERC20(collateral).safeTransfer(
      LibManager.transferRecipient(collatInfo.managerData.config),
      amountOut
   );
   LibManager.invest(amountOut, collatInfo.managerData.config);
}
```

If an earlier output is managed, its tokens remain directly held by the Diamond proxy instead of being transferred to its manager. If the final output is unmanaged, none of the earlier managed outputs are processed.

This is not merely a custody-location difference. LibGetters.getCollateralRatio() and LibSurplus._getCollateralBalance() use LibManager.totalAssets() for managed collateral and ignore the proxy's direct balance. Ordinary managed mint, burn, redemption, and surplus flows therefore do not account for or consume the stranded tokens.

Future reward sales also snapshot the already-stranded balance and process only a new positive delta, so another call does not automatically repair the state.

Preconditions

- A trusted seller or authorized caller executes an Odos payload producing multiple supported collateral outputs.
- At least one non-final output is configured as managed.
- The corresponding manager does not include the Diamond's directly held balance in totalAssets().

Failure scenario

1. An Odos sale returns managed collateral A and managed or

	unmanaged collateral B. 2. The loop detects A and records its delta. 3. The loop subsequently detects B and overwrites collateral and amountOut. 4. Only B is processed after the loop. 5. A remains in the Diamond proxy. 6. A is excluded from managed collateral accounting, redemptions, and surplus calculations. Impact Valid collateral becomes operationally stranded and invisible to the protocol's normal accounting. This can understate the collateral ratio, reduce quoted redemption outputs, trigger unnecessary redemption penalties, and require privileged manual recovery.
Recommendations	Since the manager feature will never be used in the Parallelizer, this issue can be acknowledged. If it is ever meant to be used, we recommend only ever executing single swaps.
Comments / Resolution	Acknowledged.

Issue_17	Multi-collateral sales can result in unexpected slippage drifts
Severity	**Low**
Description	sellRewards() supports an opaque ODOS payload that can consume multiple approved reward tokens and produce multiple collateral outputs. However, the function accepts only one scalar minAmountOut and applies it only to collateral balances that strictly increased. If one expected output collateral receives exactly zero, its balance remains unchanged and the corresponding slippage check is skipped entirely. The transaction still succeeds as long as at least one other collateral increased. The loop explicitly supports multiple output tokens.

Consequently, one successful output leg can mask another leg that consumed its reward input but returned nothing.

This does not allow ODOS to consume an existing supported collateral: any collateral balance decrease causes a revert. The affected inputs are external reward tokens previously approved to ODOS.

Root Cause

Before calling ODOS, sellRewards() records every supported collateral balance:

```
for (uint256 i; i < listLength; ++i) {
  balances[i] = IERC20(list[i]).balanceOf(address(this));
}
```

After ODOS executes, each collateral is checked as follows:

```
if (newBalance < balances[i]) {
  revert InvalidSwap();
} else if (newBalance > balances[i]) {
  hasIncreased = true;
  collateral = list[i];
  amountOut = newBalance - balances[i];

  if (amountOut < minAmountOut) {
    revert TooSmallAmountOut();
  }
}
```

The equality case:

```
newBalance == balances[i]
```

performs no validation.

The final check only requires one collateral to have increased:

```
if (!hasIncreased) revert InvalidSwap();
```

Therefore:

All expected outputs are zero:

Reverts

One output is positive and another expected output is zero:

Succeeds

The calldata does not identify expected output tokens and cannot bind a minimum to each expected output.

Preconditions

- Governance has approved multiple reward tokens to the ODOS router.
- A trusted seller submits an ODOS payload consuming multiple reward inputs.
- The payload is expected to produce multiple supported collateral outputs.
- At least one output collateral increases by at least minAmountOut.
- At least one other expected output collateral receives exactly zero.
- The ODOS call itself succeeds and does not enforce a sufficient internal per-leg minimum.
- The output-producing routes are independently executable or capable of partial success.

Failure Scenario

Assume the Parallelizer holds three reward tokens:

R1: 100 units
R2: 100 units
R3: 100 units

The ODOS payload is intended to execute:

R1 → collateral A
R2 → collateral B
R3 → collateral C

The trusted seller expects at least 90 units from each output and calls:

sellRewards(90, payload)

ODOS consumes all three reward-token inputs but returns:

A received: 100
B received: 100
C received: 0

RewardHandler processes the output balances:

Collateral A
newBalance(A) > oldBalance(A)
delta = 100
100 >= 90

The check passes and hasIncreased becomes true.

Collateral B
newBalance(B) > oldBalance(B)
delta = 100
100 >= 90

The check also passes.

Collateral C
newBalance(C) == oldBalance(C)

Neither the decrease branch nor the increase branch executes. No comparison against minAmountOut occurs.

The final check observes:

hasIncreased == true

The function therefore succeeds despite R3 having been consumed for zero C.

State After Execution
R1 consumed
R2 consumed
R3 consumed

A increased by 100
B increased by 100
C increased by 0

The transaction does not revert, so the loss associated with the failed R3 route is permanent.

Furthermore, because collateral and amountOut are overwritten for each positive output, the function returns only B's delta and performs manager handling only for B. This compounds the separate multi-output manager-accounting issue but is not required for the slippage bypass.

Impact

A partial multi-output reward sale can permanently consume one or more reward-token inputs without receiving their expected collateral outputs.

Potential consequences include:

- complete loss of an individual reward-sale leg;
- MEV or adverse-liquidity conditions selectively degrading one route while another route masks the failure;
- misleading successful execution despite an expected output being absent;
- incomplete events and return data because only positive outputs are reported;
- inability to express different minima for tokens with different decimals or economic values; and
- failure of keeper monitoring that treats a successful transaction

	as proof that every route was completed. The loss is limited to reward tokens approved to ODOS. Existing supported collateral cannot be consumed because any decrease in its direct balance reverts. Access-Control Considerations Only governance or a trusted seller can call sellRewards(). However, malicious privileged behavior is not required. A normal trusted keeper can submit a legitimate aggregator payload and rely on minAmountOut as the protocol-level execution safeguard. If one external route produces zero while another succeeds, the handler accepts the partial execution. If ODOS itself unconditionally enforces appropriate per-output minima inside every payload, the practical impact is reduced. The Parallelizer does not validate or guarantee that property. Different-Decimal Impact The single scalar minimum is also unsuitable for multiple output tokens with different decimals. For example: USDC: 6 decimals DAI: 18 decimals A raw minimum appropriate for DAI cannot meaningfully protect USDC, and vice versa. The current API therefore cannot provide consistent per-output economic slippage protection even when every output is positive. Furthermore, this system simply does not support tokens with different values.
Recommendations	Consider only ever executing swaps with single token outputs.
Comments / Resolution	Acknowledged.

SettersGovernor

The SettersGovernor contract is a privileged Diamond facet used to manage the Parallelizer's core configuration and collateral lifecycle. It supports adding and revoking collateral, configuring managers and oracles, adjusting issuance accounting, managing trusted addresses and allowances, recovering assets, updating payees and surplus parameters, and replacing the protocol's access manager.

Privileged Functions

- recoverERC20
- setAccessManager
- setCollateralManager

- changeAllowance
- toggleTrusted
- addCollateral
- adjustStablecoins
- revokeCollateral
- setOracle
- updateOracle
- setWhitelistStatus
- updatePayees
- updateSlippageTolerance
- updateSurplusBufferRatio

SettersGuardian

The SettersGuardian contract is a privileged Diamond facet used to control the Parallelizer's operational and risk parameters. It allows authorized roles to pause or unpause minting, burning, and redemptions, configure mint and burn fee curves, update the redemption penalty curve, manage whitelist entries, and set collateral-specific issuance caps.

Privileged Functions

- pause
- unpause
- setFees
- setRedemptionCurveParams
- toggleWhitelist
- setStablecoinCap

Surplus

The Surplus contract is a Diamond facet used to realize and distribute collateral value held above the Parallelizer's required backing. It converts eligible collateral surplus into tokenP through the Swapper, verifies that the remaining collateral ratio satisfies the configured surplus buffer, and distributes accumulated tokenP income among the configured payees.

Privileged Functions

- processSurplus
- Release

Issue_18	Negative mint fees can exceed the computed per-collateral surplus or disable surplus processing
Severity	**Low**
Description	LibSurplus._computeCollateralSurplus() calculates the amount of additional USDp that can be attributed to a collateral while respecting the configured surplus buffer. It then converts that stable-denominated amount into collateralSurplus using the mint oracle. The calculation does not apply the collateral's mint fee. Surplus.processSurplus() passes collateralSurplus into the ordinary Swapper mint path. Swapper applies the live mint fee and returns the

actual issuedAmount.

Root Cause

A negative mint fee increases the amount of USDp produced from the collateral amount selected by the surplus calculation.

processSurplus() enforces only a minimum USDp output. It does not limit issuedAmount to the previously computed stableSurplus.

After minting, the function validates only the global collateral ratio. It does not verify whether the selected collateral's attributed issuance still respects its local surplus buffer.

Preconditions

- The selected collateral has a negative mint fee over the relevant exposure segment.
- The local collateral surplus is smaller than the global extractable surplus.
- Other collateral provides additional global backing headroom.
- The collateral's independent stablecoin cap does not stop the mint.
- A restricted actor calls processSurplus().

Failure Scenario

Assume collateral A is worth $120 and has 100 USDp of attributed issuance. The configured surplus buffer is 110%.

At that buffer, A can support approximately 109.09 USDp. Its calculated local surplus is therefore approximately 9.09 USDp.

Collateral B provides substantial additional global surplus, so A's local limit is the binding value.

A has a negative 10% mint fee. Processing the collateral amount corresponding to 9.09 USDp produces approximately 10.10 USDp after applying the rebate.

Collateral A's attributed issuance increases to approximately 110.10

	USDp, which is above the amount A can locally support at the 110% buffer. The global post-processing check can still pass because collateral B provides enough excess backing. Impact Surplus processing can issue more USDp than the selected collateral's computed local surplus and implicitly consume safety margin belonging to other collateral. This distorts collateral-specific issuance, exposure calculations and future surplus calculations. If the global surplus cap is instead the binding value, the additional issuance generally pushes the global ratio below the required buffer and causes the entire transaction to revert. In that state, the same fee mismatch becomes a surplus-processing denial of service rather than successful overissuance.
Recommendations	Since governance ensures that surplus is only ever burned and that the mint fee is never negative, this issue can be acknowledged. However, purely from a technical standpoint, the protocol is simply incompatible with negative mint fee and surplus generation.
Comments / Resolution	Acknowledged.

Issue_19	A self-payee allocation Is recursively redistributed as fresh income
Severity	**Low**
Description	LibSetters.updatePayees() does not prevent the Parallelizer proxy itself from being configured as a payee. During distribution, LibSurplus._release() transfers the assigned amount from the proxy to itself: IERC20(address(_ts.tokenP)).safeTransfer(_payee, income); When _payee == address(this), this transfer has no net effect on the proxy's TokenP balance. The self-payee allocation therefore remains in the contract after release() completes. The next call to Surplus.release() reads the entire remaining balance as new income: uint256 income = ts.tokenP.balanceOf(address(this));

	It then redistributes that balance according to the same shares. The self-payee portion is repeatedly reduced while the other payees receive additional distributions. For example, with a 50% self-payee and a 50% external payee: Initial income: 100 First release: external receives 50; proxy retains 50 Second release: external receives 25; proxy retains 25 Third release: external receives 12.5; proxy retains 12.5 Repeated releases eventually redirect almost the entire self allocation to the other configured payees. An updatePayees() call can also distribute the retained balance using a newly configured payee set. Preconditions • The proxy is configured as one of multiple payees. • The self-payee has a nonzero share. • release() is executed repeatedly or the payee configuration is subsequently updated without intentionally preserving the retained balance. Impact A share intended to remain with the protocol is not retained as a stable allocation. It is progressively redistributed to other payees or burned if the zero address is among them, producing revenue allocation inconsistent with the configured shares.
Recommendations	Consider rejecting address(this) as payee.
Comments / Resolution	Fixed.

Issue_20	Chain-Local surplus calculations can reduce network-wide collateralization below the global buffer
Severity	**Low**
Description	Each Parallelizer determines extractable surplus using only its local collateral and locally tracked issuance. When USDp is globally fungible across several Parallelizers, a locally overcollateralized deployment can mint and release surplus even when another deployment is undercollateralized. The local post-operation check succeeds because the first deployment remains above its configured buffer, while the combined network collateral ratio falls below that same buffer. The supposedly extractable surplus does not exist from the perspective of all outstanding omnichain USDp. Root Cause LibSurplus._calculateGlobalExtractableSurplus() is described as global,

	but its inputs are local:
	
	(
	  uint64 collatRatio,
	  uint256 stablecoinsIssued,
	  ,
	  ,
	) = LibGetters.getCollateralRatio();
	
	LibGetters.getCollateralRatio() reads only:
	
	• the current Parallelizer's collateral list and balances; and
	• the current Parallelizer's normalizedStables * normalizer.
	
	It has no knowledge of other deployments':
	
	• reserves;
	• tracked issuance;
	• collateral ratios;
	• surplus operations; or
	• in-flight bridge state.
	
	The locally extractable amount is:
	
	stablecoinsIssued
	  * (collatRatio - surplusBufferRatio)
	  / surplusBufferRatio;
	
	Surplus.processSurplus() then mints that amount of TokenP through the local Swapper. Its final collateral-ratio check is also local:
	
	(uint64 collatRatio,,,,) = LibGetters.getCollateralRatio();
	
	if (collatRatio < ts.surplusBufferRatio) {
	  revert Undercollateralized();
	}
	Preconditions
	
	• At least two Parallelizers issue the same globally fungible USDp.
	• One deployment is above its surplus buffer.

- Another deployment is below that buffer or has no equivalent surplus.
- Cross-chain USDp remains mutually redeemable through BridgeableTokenP.
- An authorized operator calls processSurplus() on the overcollateralized deployment.
- Surplus payees include at least one nonzero recipient, so the minted TokenP is released rather than burned.

Failure Scenario

Assume two deployments and a network-wide target buffer of 105%.

Chain A
Collateral value: $110
Tracked issuance: $100
Local CR: 110%
Chain B
Collateral value: $100
Tracked issuance: $100
Local CR: 100%

Before surplus processing:

Global collateral: $210
Global issuance: $200
Global CR: 105%

Chain A calculates its locally extractable surplus:

maxBackable = $110 / 1.05
= $104.7619

stableSurplus = $104.7619 - $100
= $4.7619

It mints and later releases approximately 4.7619 USDp.

After processing:

	Chain A Collateral value: $110 Tracked issuance: $104.7619 Local CR: 105% The local post-operation check succeeds. Combined network Global collateral: $210 Global issuance: $204.7619 Global CR: 102.56% Although both deployments used a nominal 105% buffer, Chain A's locally valid surplus extraction reduced the network-wide collateral ratio from 105% to approximately 102.56%.
Recommendations	This issue exposes a design gap in the surplus calculation and can only be fixed by incorporating cross-chain states via messaging. A fix for this issue requires major refactoring of the codebase.
Comments / Resolution	Acknowledged.

Issue_21	Permissionless exposure changes can move surplus processing into a reverting mint-fee segment
Severity	**Low**
Description	Surplus.processSurplus() derives both collateralSurplus and minExpectedAmount without accounting for the live exposure-dependent mint fee. It then converts the surplus collateral into TokenP through the ordinary fee-bearing mint route. An unprivileged user can increase the target collateral's exposure and **move the mint into a fee segment whose effective fee exceeds the static surplus slippage tolerance**.

	The inner mint then reverts against processSurplus()'s own minExpectedAmount. Surplus sizing performs: stableSurplus = maxBackable - stablesBacked; collateralSurplus = convertDecimalTo((stableSurplus * BASE_18) / mintValue, 18, collatInfo.decimals, Floor); The minimum output is: minExpectedAmount = stableSurplus * (BASE_9 - slippageTolerance) / BASE_9; Processing then calls: issuedAmount = ISwapper(address(this)).swapExactInput(collateralSurplus, minExpectedAmount, collateral, address(ts.tokenP),

	address(this), block.timestamp); The mint route applies: amountOut = amountIn * BASE_9 / (BASE_9 + fees); The fee comes from _quoteFees() and depends on: currentExposure = collatInfo.normalizedStables * BASE_18 / ts.normalizedStables; Issue and Root Cause processSurplus() assumes that its configured slippageTolerance sufficiently covers the difference between calculated gross surplus and actual TokenP output.
Recommendations	Consider adjusting the slippage if this situation happens.
Comments / Resolution	Acknowledged.

Swapper

The Swapper contract is the Parallelizer's primary user-facing Diamond facet for minting tokenP against supported collateral and burning tokenP to withdraw collateral. It supports exact-input and exact-output operations, Permit2 and EIP-3009 authorization flows, oracle-based conversions, exposure-dependent fees, issuance caps, managed collateral strategies, whitelist enforcement, and normalized issuance-accounting updates.

Privileged Functions

- none

Issue_22	userDeviation can be abused to steal from the protocol
Severity	**Medium**
Description	When a collateral's spot remains inside userDeviation, readSpotAndTarget() replaces its spot with the target price for both mint and burn calculations. A user can therefore buy the collateral below target, mint TokenP at the target price, and burn the TokenP into a healthy collateral. Unlike the oracle-front-running issue, this route remains available after the oracle has already reported the depeg. Root Cause The oracle value is snapped to target when: targetPrice * (BASE_18 - deviation) < oracleValue * BASE_18 &&

oracleValue * BASE_18
< targetPrice * (BASE_18 + deviation)

If X satisfies this test:

- readMint(X) prices X at target;
- readBurn(X) also observes target;
- X contributes no depeg haircut to the global burn ratio.

The user can therefore mint against X at target and select healthy Y as burn output.

Preconditions

- X trades below target but remains strictly inside userDeviation.
- X is available externally at its discounted market price.
- X minting and Y burning are active.
- Combined mint and burn fees are smaller than the discount.
- X has remaining mint cap.
- Y has sufficient tracked issuance and liquid reserves.
- Exposure-dependent fees have not yet eliminated the profitability.

For the exact example spot = $0.99, userDeviation must be greater than 1%. A deviation of exactly 1% does not snap because the lower-bound comparison is strict.

Failure Scenario

Assume:

X market/oracle price: $0.995
X target price: $1.000
userDeviation: 1.00%
Fees: 0%

The spot lies strictly inside the permitted band.

The user acquires 100 X for $99.50 and mints:

100 X → 100 TokenP

	Because readBurn(X) also snaps X to target, it contributes a ratio of 1. The user burns into Y: 100 TokenP → 100 Y worth $100 After the cycle: • X reserves and X-attributed issuance increase; • Y reserves and Y-attributed issuance decrease; • total TokenP supply returns to its initial value; • aggregate reserve quality decreases. Impact Users can repeatedly rotate discounted collateral into healthy reserves, transferring the tolerated depeg to the protocol. The loop continues until mint caps, collateral-specific tracked issuance, liquidity, or the fee curves make the operation unprofitable. The behavior may be defended as intentional peg support, but economically it causes the Parallelizer to purchase discounted collateral at target value. It is additionally notable that the deviation is also in favor of the user during burn. If a token is worth 1.01 USD and still inside the deviation, the user will only require to burn 1 USD for it.
Recommendations	Consider setting fees in such a way that userDeviation cannot be exploited.
Comments / Resolution	Fixed.

Issue_23	Insolvent Parallelizers continue minting at face value, allowing legacy holders to capture new backing
Severity	**Medium**
Description	Swapper mint pricing does not consider the Parallelizer's global collateral ratio. If legacy TokenP remains outstanding after reserves have been depleted or materially impaired, a new user can still deposit healthy collateral and receive TokenP at the ordinary oracle price. Because TokenP is fungible, existing holders can immediately burn their legacy TokenP against the newly deposited collateral. The new minter receives the expected number of TokenP but can lose the backing they just introduced. Root Cause Mint quotes use only: • the incoming collateral's mint oracle; • its exposure-dependent mint fee; and

- its stablecoin cap.

They do not query LibGetters.getCollateralRatio() or require a minimum global solvency ratio.

amountOut = oracleValue * amountIn;
amountOut = _quoteFees(...);

After minting, the collateral becomes ordinary pooled reserves. Burn authorization depends only on possession of TokenP, not on which account or chain originally introduced the backing.

A legacy holder can burn against the new collateral as long as:

- the collateral's burn is live;
- the reserve balance is sufficient; and
- the burn does not eliminate all global tracked issuance.

Failure Scenario

Assume:

Legacy TokenP outstanding: 1,000
Tracked issuance: 1,000
Usable collateral reserves: 0
Mint and burn: live
Fees: 0%

A new user deposits 100 units of healthy collateral X:

User deposits: 100 X
User receives: 100 TokenP

State becomes:

TokenP outstanding: 1,100
X reserves: 100

A legacy holder then burns 100 TokenP into X.

Because the burn amount is smaller than the global tracked issuance

	and X has sufficient liquidity, the holder can withdraw approximately all 100 X. Afterward: X reserves: 0 New depositor still owns: 100 TokenP The newly introduced collateral has recapitalized legacy TokenP rather than exclusively backing the newly minted amount. Preconditions • Existing TokenP is materially undercollateralized. • Minting and burning remain active. • A user or automated component introduces healthy collateral through a mint. • A legacy TokenP holder monitors and burns against that collateral. • Burn fees and liquidity permit profitable extraction. Complete physical reserve depletion can arise from privileged recovery or manager loss. The same behavior also applies to partial economic insolvency, including severe collateral devaluation. Impact New depositors can suffer immediate dilution or complete capture of their deposited backing. The mint transaction's minAmountOut does not protect against this because the user receives the expected TokenP quantity; the problem is the impaired value and redeemability of those tokens immediately after minting.
Recommendations	This issue can only occur during manager loss (legitimate) and privileged recovery (governance). Since there is no manager execution planned, this issue can be acknowledged.

	Fixing it would require refactoring calculations.
Comments / Resolution	Acknowledged.

Issue_24	Chain-Local negative-fee validation does not prevent cross-chain mint-to-burn arbitrage
Severity	**Medium**
Description	LibSetters.checkFees() validates negative mint and burn fees against only the collateral fee curves stored in the current Parallelizer. This proves that an atomic mint-to-burn cycle is nonprofitable only within that deployment. Because principal USDp can be bridged to another deployment, a user can combine: • a subsidized negative-fee mint on chain A; with • a lower-fee burn on chain C. Neither setter can observe or validate the other chain's fee configuration. Root Cause When setting a negative mint fee, the local validation checks: for each local collateral c: burnFee_c + negativeMintFee >= 0

Specifically:

```
if (burnFees[0] + yFee[0] < 0) {
  revert InvalidNegativeFees();
}
```

The loop iterates only:

ts.collateralList

No cross-chain state, remote fee schedule or bridge configuration is included.

Each deployment can therefore be internally safe while their combined fee configurations are unsafe.

Preconditions

- Chain A permits a negative mint fee.
- All Chain A burn fees are sufficiently high for the local setter validation to pass.
- Chain C has a lower burn fee.
- USDp can be bridged from A to C.
- The combined bridge and execution costs are smaller than the cross-chain fee spread.
- A has remaining mint capacity.
- C has sufficient tracked issuance and collateral liquidity.
- Applicable bridge debit and credit limits are not exhausted.

Failure Scenario

Assume all collateral prices are $1.

Chain A configuration
Mint fee: -1%
Burn fee: +1%

The local validation passes:

+1% + (-1%) = 0%

Depositing $100 on A mints:

100 / 0.99 = 101.0101 USDp

A same-chain burn applies the 1% burn fee:

101.0101 × 0.99 = $100

No local profit exists.

Chain C configuration
Burn fee: 0%

The user bridges the 101.0101 USDp from A to C and burns it there:

101.0101 USDp → $101.0101 collateral

Profit before bridge and execution costs:

$101.0101 - $100 = $1.0101

Both deployments individually satisfy their local setter validations.

Impact

An unprivileged user can repeatedly:

- consume negative mint subsidies on one chain;
- bridge the enlarged USDp amount; and
- withdraw healthy collateral from another chain.

The drain is bounded by:

- the subsidized collateral's mint cap;
- bridge limits;
- destination tracked issuance;
- destination liquidity; and
- exposure-dependent fee progression.

Depending on the configured negative fee, the amplification can be considerably larger than 1%.

	This issue can be combined with the mint-fee bypass.
Recommendations	Consider not allowing for negative mint fee settings.
Comments / Resolution	Acknowledged.

Issue_25	Managed burn settlement ignores `maxAvailable()`
Severity	**Low**

Description	Burn execution calls IManager.release() for the full quoted collateral output without bounding it by IManager.maxAvailable().
	
	Although quote functions check maxAvailable(), _swap() itself does not. A tokenP holder can consume the manager's available liquidity through burns, after which basket redemptions continue to be sized from totalAssets() and revert when they attempt to release unavailable assets.
	
	Root Cause
	
	if (collatInfo.isManaged > 0) {
	
	LibManager.release(tokenOut, to, amountOut, config);
	
	}
	
	No per-caller limit or live maxAvailable() bound exists at execution.
	
	Impact
	
	A sufficiently funded holder can time the exhaustion of immediate manager liquidity and cause subsequent burns or redemptions to

	revert.
Recommendations	Since manager operations are disregarded for Parallel, this issue can be safely acknowledged.
Comments / Resolution	Acknowledged.

Issue_26	Burn firewall Is blind to non-price reserve losses
Severity	**Low**
Description	The burn firewall discounts burns only when an oracle reports that a collateral's price fell relative to its target. It does not consider whether reserves were lost, stolen, frozen or rendered illiquid while the token price remained stable, which can regularly happen in case of manager operations. After a non-price reserve loss, early tokenP holders can therefore burn at the unchanged price and withdraw the remaining liquid reserves at face value. Later holders bear the entire shortfall. Root Cause readBurn() derives its ratio exclusively from: spot price / target price Neither readBurn() nor getBurnOracle() incorporates: • the collateral's current balance; • its attributed issuance; • IManager.maxAvailable(); • system collateral ratio; or • a reserve-loss factor.

	Impact The firewall fails during exactly the class of incidents where collateral quantity disappears but the external price remains close to par. The system becomes a first-exit-wins queue: early burners receive full value until the available reserve is depleted, while later burns revert.
Recommendations	Since manager operations are disregarded in Parallel, this issue can be safely acknowledged.
Comments / Resolution	Acknowledged.

Issue_27	An economically insignificant depegged collateral applies an unweighted haircut to every direct burn
Severity	**Low**

Description	LibOracle.getBurnOracle() computes the burn ratio as the minimum ratio observed across every listed collateral:
	
	`if (ratioObserved < minRatio) {`
	`  minRatio = ratioObserved;`
	`}`
	
	It does not weight the ratio by:
	
	• collateral balance;
	• normalized issuance;
	• share of the basket;
	• whether minting or burning is paused; or
	• whether the collateral has any current economic exposure.
	
	A collateral representing 0.5% of the basket can therefore impose the same protocol-wide burn haircut as one representing 90% of the basket.
	
	Root Cause

The burn mechanism models basket safety using the worst individual collateral ratio rather than aggregate backing.

The call path is:

Swapper._quoteBurnExactInput()
→ LibOracle.getBurnOracle()
→ readBurn() for every collateral
→ minimum ratio
→ healthy collateral output multiplied by minimum ratio

This is explicitly implemented as a firewall against withdrawing healthy collateral while leaving impaired collateral behind. The disproportionality arises because the firewall contains no exposure weighting or inactive-collateral filtering.

Preconditions

- Collateral X remains in collateralList.
- X trades below its target by more than burnRatioDeviation.
- X represents only a small portion of normalized issuance or reserves.
- A user attempts to burn TokenP for healthy collateral Y.
- Y's burn path remains active and has sufficient issuance and liquidity.

The effect can persist even if X's own minting or burning is paused. A stronger boundary exists if X has zero normalized issuance but has not yet been revoked.

Failure Scenario

Assume:

Total tracked issuance: 100 million TokenP
X exposure: 0.5 million (0.5%)
Y exposure: 99.5 million
X price: $0.50
Y price: $1.00

The aggregate collateral value is approximately:

0.5m × $0.50 + 99.5m × $1
= $99.75m

The basket's weighted collateral ratio is therefore approximately:

99.75%

Nevertheless, getBurnOracle() selects:

minRatio = 50%

A user burning 1 million TokenP for Y receives approximately:

1m × 50% / $1 = 0.5m Y

rather than an amount close to the basket's weighted 99.75% backing.

The user burns the full 1 million TokenP while only 0.5 million Y leaves the reserves. The retained collateral recapitalizes the system at the exiting user's expense.

Impact

Direct burns into every healthy collateral can become heavily impaired even though aggregate undercollateralization is minimal. This can:

- suppress ordinary exit liquidity;
- create external TokenP peg pressure;
- incentivize users to race toward redemption or secondary-market exits;
- make pausing the impaired collateral insufficient to restore healthy burns; and
- recapitalize the protocol disproportionately through early direct burners.

There is no direct attacker-profit objective. The finding is primarily an economic-liveness and fairness concern, and the protocol can reasonably argue that it is the intended conservative behavior of the

	burn firewall.
Recommendations	This issue is inherent from the design of the protocol and fixing it would require refactoring the burn math. We do not recommend this.
Comments / Resolution	Acknowledged.

Issue_28	Oracle-update front-running can convert stale-priced collateral into healthy reserves within the burn firewall band
Severity	**Low**
Description	A user can mint immediately before a downward oracle update using the old higher collateral price and burn after the update into a healthy collateral. This is usually a pattern which is prevented in future protocols as they use an init <-> execute approach to prevent immediate profit. The burn firewall prevents the extraction only when the new collateral price falls below the burnRatioDeviation boundary. If the new price is outside userDeviation but remains inside burnRatioDeviation, the updated spot price is recognized for minting, but the depeg is deliberately omitted from the basket-wide burn ratio. This creates a front-running window around downward oracle updates, especially since chainlink oracle updates are trivial front-runnable. Root Cause Minting uses: oracleValue = LibOracle.readMint(collatInfo.oracleConfig); Before the update, the stale spot remains at the target, so the user

receives full-value TokenP.

Burning subsequently uses:

```
(uint256 ratio, uint256 oracleValue) =
    LibOracle.getBurnOracle(collateral, collatInfo.oracleConfig);
```

For each collateral, readBurn() only lowers the basket ratio when:

oracleValue < targetPrice * (1 - burnRatioDeviation)

If:

userDeviation < depeg ≤ burnRatioDeviation

then:

- the updated price would no longer receive the mint-side target snap;
- but the burn-side basket ratio remains 1;
- healthy collateral can still be withdrawn at full value.

The strict < comparison means a price exactly at the lower burn-deviation boundary also avoids the haircut.

Preconditions

- userDeviation < burnRatioDeviation.
- A collateral's market price has already decreased, but the configured oracle has not yet incorporated the decrease.
- The downward oracle update is observable or predictable sufficiently early for transaction ordering.
- The post-update price remains within the burn firewall's tolerated band.
- Combined mint and burn fees, gas, and execution costs are smaller than the price discrepancy.
- The target healthy collateral has sufficient normalized issuance and available reserves.
- Mint and burn operations remain active.

Failure Scenario

Assume:

Target price: $1.00
Old oracle price: $1.00
New market/oracle: $0.995
userDeviation: 0.20%
burnRatioDeviation: 1.00%
Fees: 0%

Before the oracle update, the user acquires 100 units of collateral X for approximately $99.50 and mints using the stale $1 price:

100 X → 100 TokenP

After the oracle update:

- X's $0.995 price is outside the 0.20% user-deviation band;
- but it is inside the 1% burn-ratio band;
- readBurn(X) therefore reports a basket ratio of 1.

The user burns the 100 TokenP for healthy collateral Y:

100 TokenP → 100 Y worth $100

The gross gain is approximately:

$100 - $99.50 = $0.50

The protocol's reserves have effectively purchased X at its stale $1 valuation and released Y at its current $1 valuation.

Impact

A user can transfer the loss from a newly depegging collateral to the Parallelizer's healthy reserves. The extractable value is bounded per unit by the tolerated deviation and reduced by fees, but it can be scaled up to the available mint cap, favorable fee-curve region, and healthy collateral liquidity.

This path is generally one-shot per downward oracle movement. It

	becomes repetitive if there are multiple predictable downward updates or if the spot remains inside userDeviation, which overlaps with the previously discussed steady-state circular reserve drain.
Recommendations	Consider setting the fees to values which prevent this extraction.
Comments / Resolution	Acknowledged.

Issue_29	Temporary cross-collateral issuance can suppress exposure-based fees
Severity	**Low**
Description	A user can temporarily mint USDp against collateral B to reduce collateral A's relative issuance exposure, mint against A while A is charged a lower exposure-based fee, and then burn the temporary USDp against B to restore the original B position. The final collateral exposure is equivalent to directly minting against A, but the user can pay a materially smaller fee. Root Cause Swapper._quoteFees() calculates the current exposure using: • the issuance attributed to the collateral being exchanged; and • the current global Parallelizer issuance. The calculation therefore trusts the instantaneous, transaction-local distribution of issuance. It does not distinguish persistent issuance from issuance temporarily created through another collateral. Each successful mint immediately increases both the selected collateral's normalizedStables and global normalizedStables. A later burn can reverse the temporary collateral's issuance after the advantageous mint has already been priced. Preconditions

- Collateral A has a non-flat mint-fee curve that charges more at higher exposure.
- Collateral B can be minted and subsequently burned at a round-trip cost smaller than the fee avoided on A.
- Collateral B has sufficient stablecoin cap.
- The attacker can temporarily source B
- B has enough attributed issuance after the temporary mint for the corresponding burn not to underflow.

Attack Path

1. Collateral A is currently highly exposed and would incur a high mint fee.
2. The user temporarily deposits collateral B and mints a large amount of USDp.
3. This increases global tracked issuance while leaving A's attributed issuance unchanged, reducing A's reported relative exposure.
4. The user mints against A while `_quoteFees()` evaluates A from this diluted exposure.
5. The user burns the temporarily minted USDp for B and recovers the temporary B position.
6. The final state contains the intended additional A exposure, but the corresponding A mint paid a fee calculated from a temporary intermediate state.

For example, if A accounts for 80 of 100 issued USDp, it begins at 80% exposure. Temporarily minting 100 USDp against B lowers A's exposure to 40%. A subsequent mint against A can therefore remain in a low-fee region. Burning 100 USDp for B restores B's original position, while the lower fee charged on the A mint remains final.

Impact

Users can bypass the intended economic cost of increasing concentration in a heavily exposed collateral. The protocol loses fees and can accumulate riskier collateral more cheaply than governance

	intended. The stablecoin cap still limits absolute issuance, so this does not bypass the hard cap itself.
Recommendations	This is a game-theoretical issue which only becomes material once the fees do not offset the savings. Therefore, we recommend setting fees to such an extent to prevent these scenarios.
Comments / Resolution	Acknowledged.

Issue_30	Burn-fee segment selection can underflow after independently rounded de-normalization
Severity	**Low**
Description	Swapper._quoteFees() determines the active burn-fee segment using normalized issuance values. It then calculates the distance from the selected breakpoint using separately de-normalized and rounded issuance amounts. Near a burn-fee breakpoint, these two representations can disagree by one raw TokenP unit. The normalized exposure places the collateral below the breakpoint, but the de-normalized amounts make its current issuance one unit larger than the amount associated with that breakpoint. The function then attempts to subtract the larger current issuance from the smaller breakpoint amount. Because both values are unsigned integers, the subtraction underflows and reverts with Panic(0x11). While the state remains in this rounding window, every burn quote and burn transaction for the affected collateral reverts. Context The issue occurs in Swapper._quoteFees().

	The function first calculates exposure using the normalized accounting counters: currentExposure = uint64((collatInfo.normalizedStables * BASE_18) / normalizedStablesMem); It then independently converts the collateral and global issuance into actual stablecoin amounts: v.stablecoinsIssued = (uint256(collatInfo.normalizedStables) * normalizerMem) / BASE_27; v.otherStablecoinSupply = (normalizerMem * normalizedStablesMem) / BASE_27 - v.stablecoinsIssued; The active burn segment is selected using the normalized exposure:

	uint256 i = LibHelpers.findLowerBound(false, collatInfo.xFeeBurn, uint64(BASE_9), uint64(currentExposure)); Inside a nonconstant segment, the function calculates how far the current issuance is from the preceding breakpoint: uint256 amountFromPrevBreakPoint = (v.otherStablecoinSupply * v.lowerExposure) / (BASE_9 - v.lowerExposure) - v.stablecoinsIssued; The final subtraction can underflow. Issue and Root Cause The segment selector and the breakpoint-distance calculation operate in different arithmetic domains: • segment selection uses the original normalized counters; • breakpoint sizing uses actual issuance amounts obtained after applying normalizer; • the collateral issuance and total issuance are rounded down independently; • otherStablecoinSupply is calculated from the independently rounded values.

The selected segment is therefore not guaranteed to remain mathematically valid after de-normalization.

When exposure is comfortably inside a segment, the rounding difference is insignificant. At the exact breakpoint boundary, however, the normalized comparison may have a safety margin of only one internal unit. Independent floor rounding can reverse that one-unit relationship.

The code performs no validation before subtracting:

breakpointAmount - stablecoinsIssued

It assumes the first value must be greater than or equal to the second because of the earlier segment selection. That assumption is false after the independent rounding operations.

Verified Numerical Example

The following burn-fee curve satisfies the validation performed by LibSetters.checkFees():

Exposure breakpoints: 100%, 70%, 50%

Burn fees: 0%, 0%, 0.5%

Assume the protocol reaches the following state:

Value	Raw amount
normalizer	1000182381996942413955249209
Total normalized issuance	1000001200762169669996 72

	46
Collateral normalized issuance	70000084053351876899770 72
Selected breakpoint	700000000
Calculated normalized exposure	699999999999999999

The calculated exposure is one internal unit below the 70% breakpoint:

Calculated exposure: 699999999999999999

70% breakpoint: 700000000000000000

The segment selector therefore chooses the segment beginning at 70%.

After separately applying the normalizer and rounding down, _quoteFees() obtains:

Current collateral issuance:

7,001,285,080,846,766,404,165,054

Issuance implied by the selected breakpoint:

7,001,285,080,846,766,404,165,053

The breakpoint-derived value is one raw unit smaller than the current issuance. The contract attempts:

7,001,285,080,846,766,404,165,053

	- 7,001,285,080,846,766,404,165,054 The mathematical result is -1. Since the destination variable is a uint256, the transaction reverts with Panic(0x11). The same state with normalizer == BASE_27 does not underflow because the normalized and actual issuance domains coincide exactly. This confirms that independently rounded de-normalization is the cause.
Recommendations	Since the mathematical architecture is highly complex and this issue only occurs in extremely rare cases, we recommend acknowledging it.
Comments / Resolution	Acknowledged.

Issue_31	Economically irrelevant collateral can impose a system-wide burn haircut and become difficult to revoke
Severity	**Low**
Description	Every Swapper burn uses the lowest burn ratio reported by any collateral in collateralList. The minimum is not weighted by: • collateral balance; • reserve value; • attributed issuance; • mint/burn status; or • economic relevance. A collateral with virtually zero reserves and only normalized-issuance dust can therefore impose its depeg haircut on burns into every healthy collateral. The problem is amplified because governance cannot revoke the collateral while any normalizedStables remains. Normalizer rounding can

	make the final normalized units impossible to remove using integer-denominated TokenP adjustments.
	
	Root Cause
	
	LibOracle.getBurnOracle() loops through every supported collateral:
	
	for (uint256 i; i < collateralList.length; ++i) {
	(, ratioObserved) =
	readBurn(ts.collaterals[collateralList[i]].oracleConfig);
	
	if (ratioObserved < minRatio) {
	minRatio = ratioObserved;
	}
	}
	
	There is no materiality check before incorporating ratioObserved.
	
	The resulting minimum ratio is applied to burns into any target collateral:
	
	amountOut =
	convertDecimalTo(
	amountAfterFee * minRatio / targetOracleValue,
	...
	);
	
	Separately, revocation requires exact zero:
	
	if (collatInfo.normalizedStables > 0) {
	revert CollateralBacked();
	}
	
	Both user burns and governance adjustStablecoins() convert absolute TokenP to normalized units using floor division:
	
	normalizedAmount =
	amount * BASE_27 / normalizer;
	
	When normalizer < BASE_27, the smallest positive TokenP amount can

	correspond to several normalized units. A residual normalized amount smaller than that minimum decrement cannot be removed without underflow. Preconditions • Collateral X remains in collateralList. • X's oracle price is below its burnRatioDeviation threshold. • X has economically negligible reserves or issuance. • X.normalizedStables remains nonzero. • For the revocation lock, the dust is smaller than the minimum normalized decrement representable by one TokenP base unit. • Users burn TokenP into another, healthy collateral. Failure Scenario Assume: Healthy collateral H: Reserve value: $10,000,000 Oracle ratio: 1.00 Legacy collateral X: Reserve value: approximately $0 normalizedStables: 1 unit Oracle ratio: 0.20 Global normalizer: 0.40e27 For one TokenP base unit, the normalized decrement is: floor(1 × 1e27 / 0.40e27) = 2 X contains only one normalized unit. Therefore: • burning one TokenP unit against X tries to subtract two normalized units and reverts; • adjustStablecoins(X, 1, false) also tries to subtract two and

	reverts; • using zero TokenP produces no adjustment; and • revokeCollateral(X, ...) reverts because one normalized unit remains. Meanwhile, getBurnOracle() observes X's 20% ratio and uses it globally. A user burning 100 TokenP into healthy collateral H receives approximately: 100 × 0.20 = $20 of H before fees, despite X having negligible economic relevance. Impact • A dust collateral can impose a severe haircut on every Swapper burn. • Healthy collateral exits become uneconomical. • The effect persists even if X minting and burning are paused. • Governance revocation can be blocked by normalized dust. • Governance may need to manipulate the oracle configuration, alter the global normalizer, or upgrade the implementation to remove X's influence. • If X's oracle reverts rather than returning a low ratio, the same unconditional loop can revert every burn quote and execution. Redemption is a separate route and uses value-weighted collateral balances, so it may remain available. The practical severity increases when redemption is paused, penalized, or operationally unsuitable.
Recommendations	Consider implementing an emergency collateral removal path which ignores the normalizedStables accounting. This should only be invoked upon absolute emergency cases and may expose unexpected side-effects.
Comments / Resolution	Acknowledged.

LibAuthorization

The LibAuthorization library derives operation-specific EIP-3009 nonces for authorized swaps and redemptions. It binds the signed token transfer to the complete intended action, including the participating tokens, amounts, recipient, slippage constraints, deadline, forfeited assets, and user-provided salt, preventing the authorization from being reused with materially different execution parameters.

Privileged Functions

- none

Issue_32	Non-standard EIP3009 tokens may be replayed due to lack of chainId incorporation into nonce
Severity	**Low**
Description	LibAuthorization.computeSwapExactInputNonce(), computeSwapExactOutputNonce(), and computeRedeemNonce() do not

include:

block.chainid
address(this)

The protocol instead relies on the token's EIP-3009 implementation to provide chain and token-contract separation through its EIP-712 domain. This is correct for standard implementations, but a collateral advertised as EIP-3009-compatible that omits chainId could leave the Parallelizer's complete swap intent replayable on another chain.

Root Cause

The Parallelizer-generated nonce commits to the operation's parameters, but not its execution domain:

nonce = hash(action type, operation parameters, userSalt)

It is therefore identical across chains when:

- the same arguments are used;
- the relevant token and recipient addresses are identical; and
- the user provides the same userSalt.

Ordinarily, this is safe because the nonce is embedded inside a signed EIP-3009 authorization whose EIP-712 domain commits to chainId and verifyingContract.

The remaining contract binding is also provided by the signed EIP-3009 to field because _executeAuthorization() requires the authorization to transfer tokens to the executing Parallelizer:

```
receiveWithAuthorization(
    params.from,
    address(this),
    params.value,
    ...
);
```

Consequently, the Parallelizer address must generally also be identical

across the affected chains.

Preconditions

- A configured collateral supports receiveWithAuthorization(), but its domain does not bind chainId.
- The token contract occupies the same address across two chains, or its domain also fails to bind verifyingContract.
- The Parallelizer occupies the same address on both chains because the signed transfer recipient is address(this).
- The user owns sufficient collateral on both chains.
- The same authorization nonce remains unused in the destination chain's independent authorizationStates mapping.
- The EIP-3009 validity window and Parallelizer deadline remain open.

No concrete configured collateral satisfying these conditions has been identified, so this is merely a theoretical scenario.

Failure Scenario

A user authorizes a collateral-to-TokenP mint on chain A:

from: user
amount: 1,000 collateral
receiver: payment recipient
userSalt: S

The derived nonce is identical on chain B because it does not contain the chain ID or Parallelizer address.

After the authorization executes on chain A, a relayer submits the same operation on chain B. The token's authorization state is independent on each chain, so the nonce remains unused on B. If the non-standard token verifies the same digest, the second execution:

- withdraws another 1,000 collateral from the user on B;
- mints another corresponding amount of TokenP; and
- sends it to the originally signed receiver.

	The relayer cannot freely change the receiver because it is committed inside the Parallelizer nonce. Impact The same signed intent can execute on multiple chains, causing an unauthorized duplicate conversion of the signer's collateral. The impact is strongest when the original receiver is a third-party payment recipient, because that recipient receives the output twice while the signer is charged twice. This is not currently a confirmed protocol finding because it depends on integrating a materially non-standard EIP-3009 token and matching cross-chain addresses.
Recommendations	Consider evaluating each collateral token carefully.
Comments / Resolution	Acknowledged.

LibDiamond

The LibDiamond library provides the core EIP-2535 upgrade and access-control functionality used by the Parallelizer's Diamond proxy. It adds, replaces, and removes facet selectors, executes optional initialization delegatecalls, validates facet bytecode, checks governor roles, and processes immediate or scheduled permissions through the configured access manager.

Privileged Functions

- none

Issue_33	Delayed authority probe blocks the trusted-caller fallback
Severity	**Low**
Description	Two functions authorize callers through: AccessManager permission OR local trusted mapping but evaluate checkCanCall() first. checkCanCall() does not merely return a boolean. For a caller holding the selector role with a nonzero delay, it attempts to consume a scheduled operation and reverts if none is ready. The local trusted mapping is therefore never evaluated. Failure Scenario 1. An address is locally trusted through isTrusted or isSellerTrusted. 2. The same address also receives a delayed AccessManager role for the selector.

	3. It calls the function without a scheduled operation. 4. checkCanCall() enters consumeScheduledOp() and reverts. 5. The independent local trust authorization is unreachable. Impact Granting an additional delayed role removes an existing immediate permission. Trusted updater and seller automation may unexpectedly stop, blocking normalizer reconciliation or reward sales.
Recommendations	Consider acknowledging this issue.
Comments / Resolution	Acknowledged.

Issue_34	Delayed governor membership Is treated as immediate authority
Severity	**Low**

Description	Negative fee curves require governor authorization because they can transfer protocol value to users. The authorization helper checks only whether the caller is a governor-role member and discards the role's execution delay.
	
	Root Cause
	
	function isGovernor(address caller) internal view returns (bool) {
	
	(bool isMember,) =
	
	accessManager.hasRole(GOVERNOR_ROLE, caller);
	
	return isMember;
	
	}
	
	A caller with delayed GOVERNOR_ROLE is therefore treated as an

	immediate governor when checkFees() validates negative fees. Preconditions The caller must: • hold delayed GOVERNOR_ROLE; and • separately possess immediate authority to call setFees. Impact The caller can install protocol-funded negative fees without scheduling the governor operation or waiting for its intended delay.
Recommendations	Consider acknowledging this behavior and making sure that the governor is always fully trusted.
Comments / Resolution	Acknowledged.

LibDiamondEtherscan

The LibDiamondEtherscan library maintains a dummy implementation address used to support verification and implementation discovery for the Diamond proxy on block explorers such as Etherscan. It stores the designated implementation in a dedicated storage slot and emits an upgrade-compatible event whenever that address is changed.

Privileged Functions

- none

No issues found.

LibGetters

The LibGetters library computes the Parallelizer's aggregate collateralization state for use by getter and redemption flows. It collects directly held collateral and externally managed sub-collateral balances, values each collateral through its redemption oracle, calculates the total tokenP issuance and collateral ratio, and returns the token ordering required for proportional redemptions.

Privileged Functions

- none

Issue_35	Manager balance arrays can diverge from configured token arrays, causing wrong-token releases or global reverts
Severity	**Low**
Description	For managed collateral, the Parallelizer stores token identities in: managerData.subCollaterals The external manager independently returns only a balance array: IManager.totalAssets() → (uint256[] balances, uint256 totalValue) LibGetters.getCollateralRatio() zips the returned balances against the configured token array by index. It does not validate that: • both arrays have equal lengths; • both arrays use the same ordering; or

- totalValue is consistent with those balances.

Length or order divergence can attach balances to the wrong token, invoke the wrong manager during redemption, or revert every collateral-ratio-dependent operation.

Root Cause

The output arrays are allocated using configured lengths:

```
subCollateralsAmount +=
   collateral.managerData.subCollaterals.length;
```

But filling and index advancement use the external manager's returned length:

```
uint256 numSubCollats = subCollateralsBalances.length;

for (uint256 k; k < numSubCollats; ++k) {
   tokens[countCollat + k] =
      address(collateral.managerData.subCollaterals[k]);

   balances[countCollat + k] =
      subCollateralsBalances[k];
}

countCollat += numSubCollats;
```

Meanwhile, subCollateralsTracker continues to reflect configured lengths. Redeemer later uses that tracker to decide which manager owns each token:

```
Collateral storage collatInfo =
   ts.collaterals[collateralList[indexCollateral]];

LibManager.release(
   redemption.tokens[i],
   redemption.to,
   redemption.amounts[i],
```

collatInfo.managerData.config
);
Preconditions

- Managed collateral is enabled.
- Governance configures a manager and a subCollaterals array.
- The manager returns a different number or ordering of balances.
- Redemption, collateral-ratio calculation, or surplus processing consumes the result.

The divergence can result from a manager upgrade, integration bug, configuration error, or changed internal asset ordering.

Failure Scenario: Shorter Returned Array

Assume:

Managed collateral M configured tokens: [M, A]
Next collateral: N
Configured managed length: 2
Manager returned balances length: 1

LibGetters constructs:

tokens[0] = M
tokens[1] = N
tokens[2] = address(0)

But subCollateralsTracker still says indices 0 and 1 belong to M.

During redemption:

1. Index 0 releases M through M's manager.
2. Index 1 contains token N.
3. The tracker still identifies index 1 as belonging to M.
4. Redeemer calls M's manager:

MManager.release(N, receiver, amountN)

The manager may:

- release unrelated N reserves;
- interpret the token unexpectedly; or
- revert and block redemption.

Failure Scenario: Longer Returned Array

If the manager returns more balances than configured tokens, this access eventually exceeds the configured array:

managerData.subCollaterals[k]

The call reverts. Because getCollateralRatio() reads every managed collateral, the result is a global redemption and collateral-ratio outage.

Failure Scenario: Same Length, Different Order

If configured tokens are:

[M, A]

but balances are returned internally as:

[balanceA, balanceM]

the Parallelizer treats balanceA as M and balanceM as A. Quotes, minimum outputs, and manager release amounts are calculated for the wrong assets.

Impact

When enabled, the mismatch can cause:

- wrong-token redemption attempts;
- excess release requests for one sub-collateral;
- understated or overstated user outputs;
- redemption failures;
- corrupted quote results; and
- global denial of service for all functions consuming getCollateralRatio().

Scope Conclusion

	The developer has stated that managed collateral scenarios are unused in the current and future Parallelizer implementation. Under that explicit assumption, the issue is out of scope even though the code-level mismatch is valid.
Recommendations	Since the Parallelizer does not use the managed system, this issue can be acknowledged. However, if that is ever going to be used, it needs to be ensured that the arrays match, especially notable that the order can change if some collateral is removed on the parallelizer via swap and pop.
Comments / Resolution	Acknowledged.

LibHelpers

The LibHelpers library provides shared mathematical and array-processing utilities used throughout the Parallelizer. It handles decimal conversions with explicit rounding, token membership checks, breakpoint searches across increasing or decreasing arrays, and evaluation of piecewise-linear fee and penalty curves.

Privileged Functions

- none

LibManager

The LibManager library provides the abstraction through which the Parallelizer interacts with externally managed collateral strategies. It decodes manager configurations and routes collateral deposits, withdrawals, balance queries, available-liquidity checks, and transfer-recipient resolution to the configured external manager.

Privileged Functions

- none

LibOracle

Privileged Functions

- none

Issue_36	Oracle ratchet compares values from different base domains
Severity	**Low**

Description	updateOracle() reads spot using BASE_18 as the oracle's base value:
	
	read(oracleType, BASE_18, oracleData);
	
	Normal pricing consumers instead calculate the stored target first and pass that target as the base:
	
	targetPrice = read(targetType, BASE_18, targetData);
	
	oracleValue = read(oracleType, targetPrice, oracleData);

	For oracle modes whose result scales with baseValue, these two calls return values in different domains. updateOracle() then compares the unit-based result against a stored target price and may either fail to ratchet or store a value that consumers subsequently scale again. Preconditions This issue requires a MAX target combined with a base-dependent spot configuration, principally: • CHAINLINK_FEEDS using a non-UNIT quote mode; or • another oracle mode that returns or propagates baseValue. Oracle modes that ignore baseValue are unaffected. Depending on the values: • the target may remain frozen because the cross-domain comparison never succeeds; or • a ratio-like value may be stored as a price and subsequently multiplied through again. The resulting target can distort mint pricing and the global burn firewall.
Recommendations	Consider ensuring that the settings align.
Comments / Resolution	Acknowledged.

LibSetters

The LibSetters library implements the validated storage transitions used by the Parallelizer's governor and guardian configuration facets. It manages collateral onboarding and removal, external managers, issuance accounting, oracle and whitelist configuration, pause states, fee and redemption curves, trusted addresses, stablecoin caps, payee distributions, and surplus-processing parameters.

Privileged Functions

- nonw

Issue_37	Non-atomic collateral onboarding inserts an oracle-less entry that reverts every basket-wide oracle read
Severity	**Low**
Description	Runtime collateral onboarding is split between multiple restricted entry points: addCollateral() setOracle()

	setFees()
	setStablecoinCap()
	unpause()
	
	addCollateral() immediately pushes the collateral into collateralList, but its oracleConfig remains empty until a subsequent setOracle() call.
	
	Both Swapper burns and redemptions iterate through every entry in collateralList. Encountering the new collateral causes LibOracle to decode an empty oracle configuration and revert. This disables exits involving every existing collateral until configuration is completed.
	
	The relevant atomicity requirement is “same transaction,” not merely “same block.” Separate transactions in the same block still expose an intermediate state and can be reordered or separated.
	
	Root Cause
	
	LibSetters.addCollateral() performs:
	
	collatInfo.decimals = uint8(IERC20Metadata(collateral).decimals());
	ts.collateralList.push(collateral);
	
	It does not require or install a valid oracle.
	
	An existing collateral burn subsequently executes:
	
	Swapper._quoteBurnExactInput()
	→ LibOracle.getBurnOracle()
	→ iterate collateralList
	→ LibOracle.readBurn(newCollateral.oracleConfig)
	→ abi.decode(empty bytes)
	→ revert
	
	Redemption similarly executes:
	
	Redeemer._quoteRedemptionCurve()
	→ LibGetters.getCollateralRatio()
	→ LibOracle.readRedemption(empty oracleConfig)
	→ revert

The new collateral being paused does not help because getBurnOracle() and getCollateralRatio() do not skip paused or zero-exposure collateral.

Preconditions

- Governance calls the runtime addCollateral() entry point.
- The valid oracle is installed through a later transaction.
- Existing burns or redemptions are live.
- The newly inserted collateral remains in collateralList with an empty oracleConfig.

The deployment initializer does not have this problem because it calls addCollateral() and setOracle() during one atomic initialization transaction.

Failure Scenario

1. Existing collaterals A and B are fully configured and active.
2. Governance calls addCollateral(C).
3. C is immediately appended to collateralList.
4. C remains paused, has zero issuance, and has an empty oracle configuration.
5. A user attempts to burn TokenP for A.
6. getBurnOracle(A) reads A, B, and then C.
7. Reading C's empty configuration reverts.
8. A redemption also reverts through getCollateralRatio().
9. The outage persists until governance successfully installs C's oracle or revokes C.

If the intended setOracle(C) transaction fails because its configuration is invalid, delayed by the access manager, or never executed, the outage persists indefinitely.

Impact

All direct burns and proportional redemptions can become unavailable during collateral onboarding. Existing collateral minting does not necessarily perform a basket-wide oracle read, so issuance can remain

	active while exits are unavailable. The same invalid entry also affects collateral-ratio getters and any surplus flow that consumes getCollateralRatio().
Recommendations	Consider executing collateral additions atomic.
Comments / Resolution	Acknowledged.

Issue_38	Enabling management excludes existing direct collateral from accounting
Severity	**Low**
Description	Changing a collateral from unmanaged to managed immediately changes the protocol's authoritative balance source: Unmanaged: collateral.balanceOf(Parallelizer) Managed: manager.totalAssets()

setCollateralManager() does not migrate or reject collateral already held directly by the Parallelizer.

Existing direct balances therefore disappear from collateral-ratio, redemption, burn-liquidity, and surplus calculations as soon as isManaged becomes true.

Root Cause

LibSetters.setCollateralManager() sets:

collatInfo.isManaged = 1;
collatInfo.managerData = managerData;

It does not:

- check IERC20(collateral).balanceOf(address(this));
- transfer that balance to the manager;
- call invest() for the existing balance; or
- require that migration was completed beforehand.

After the switch:

- LibGetters.getCollateralRatio() reads only manager.totalAssets().
- LibSurplus._getCollateralBalance() reads only manager.totalAssets().
- Swapper._checkAmounts() reads only manager.maxAvailable().
- Burns and redemptions call manager.release() rather than transferring the direct balance.

Preconditions

- An unmanaged collateral has a nonzero balance at the Parallelizer.
- Governance enables a manager for it.
- The existing balance has not separately been moved into that manager.

Failure Scenario

	1. The Parallelizer directly holds 1,000,000 units of collateral X. 2. Governance configures an empty manager for X. 3. isManaged immediately becomes true. 4. getCollateralRatio() now observes the manager's zero assets and ignores the 1,000,000 X held by the Parallelizer. 5. Redemptions quote no X from that balance. 6. Ordinary burns use manager.maxAvailable() and manager.release(), so the direct balance cannot satisfy them. 7. Surplus calculations likewise ignore the balance. The protocol can consequently appear undercollateralized even though it physically controls the collateral. Impact • Artificially reduced collateral ratio. • Incorrect redemption penalties and output calculations. • Burns can fail despite sufficient direct liquidity. • Surplus processing ignores available reserves. • Users redeeming during the transition can receive less than the reserves economically available to the system.
Recommendations	This is not an issue since collateral will not be managed in the Parallelizer. It needs to be considered if that changes and setting + deposit needs to be done atomically.
Comments / Resolution	Acknowledged.

Issue_39	A nonnegative fee update can invalidate an existing negative-fee invariant
Severity	**Low**
Description	LibSetters.checkFees() prevents the initial configuration of a negative mint or burn fee when it would permit a profitable atomic mint-to-burn cycle. However, the cross-curve check runs only when the newly

supplied curve begins with a negative fee.

A later update to a zero or positive counterpart curve skips the check, even though that update can make a previously valid negative curve unsafe.

Root Cause

The relevant validation is guarded by:

```
if (yFee[0] < 0) {
   // Compare the incoming negative curve against counterpart curves
}
```

This validates the relationship only when installing a negative curve.

It does not revalidate existing negative curves when:

- a burn curve is updated to a lower nonnegative fee while a negative mint fee exists; or
- a mint curve is updated to a lower nonnegative fee while a negative burn fee exists.

`setFees()` then overwrites the curve without checking the resulting global configuration.

Preconditions

- Governance has previously configured a negative fee.
- At installation, the opposing fees were large enough to satisfy `negativeFee + opposingFee >= 0`.
- A subsequent authorized update lowers one opposing curve to zero or another insufficient nonnegative value.
- Minting and burning remain enabled for the affected collaterals.

The second update does not itself need to be negative, so it bypasses both the invariant check and the governor-only negative-fee branch.

Failure Scenario

Initially:

Collateral A mint fee: -1%
Collateral B burn fee: +1%

This is accepted because:

-1% + 1% = 0

Depositing 100 units of A produces approximately:

100 / (1 - 1%) = 101.0101 TokenP

Burning those TokenP with the original 1% burn fee returns approximately:

101.0101 × (1 - 1%) = 100 units of B

An authorized actor later changes B's burn fee to 0%. Because the incoming curve is nonnegative, checkFees() does not compare it with A's existing -1% mint fee.

The same cycle now returns:

101.0101 × (1 - 0%) = 101.0101 units of B

The user gains approximately 1.0101 units before rounding and oracle effects.

Impact

Any user can repeatedly convert collateral A into a larger value of collateral B, extracting reserves while the system records each individual mint and burn as valid. The drain is bounded only by collateral balances, stablecoin caps, pause controls, and the regions over which the affected fee curves apply.

The fault can also arise in the symmetric direction: an existing negative burn fee becomes unsafe after a nonnegative mint-fee update.

	Notably, there is also an issue for cross-chain deployments with regards to the fee setter validation.
Recommendations	Consider being careful with any fee settings.
Comments / Resolution	Acknowledged.

LibStorage

The LibStorage library provides access to the Parallelizer's isolated storage regions. It maps the predefined storage slots to the Diamond routing state, the protocol-specific Parallelizer state, and the dummy implementation state used for block-explorer compatibility.

Privileged Functions

- none

No issues found.

LibSurplus

The LibSurplus library calculates the amount of excess collateral that may be processed while preserving the configured collateralization buffer. It accounts for direct and externally managed collateral, caps extraction by the protocol's global surplus, calculates minimum acceptable tokenP output, and distributes accumulated tokenP among payees according to their configured shares.

Privileged Functions

- none

LibWhitelist

The LibWhitelist library verifies whether an address may receive restricted collateral. It checks the Parallelizer's internal whitelist and, where configured, delegates authorization checks to an external Keyring guard.

Privileged Functions

- none

DiamondProxy

The DiamondProxy contract is the main entry point of the Parallelizer's EIP-2535 Diamond architecture. It installs the initial set of facets during deployment and routes subsequent calls by function selector through `delegatecall`, causing each facet to execute against the proxy's shared storage.

Privileged Functions

- none

Storage

The Storage module defines the enums and shared data structures used across the Parallelizer's Diamond facets and libraries. These definitions cover selector routing, collateral configuration, external managers, authorization parameters, oracle and whitelist modes, normalized issuance accounting, operational state, and surplus distribution.

Privileged Functions

- none

No issues found.

SavingsNameable

The SavingsNameable contract extends the Savings vault with governance-controlled token metadata. It stores the vault share token's name and symbol in dedicated upgrade-safe storage and allows authorized callers to update both values.

Privileged Functions

- setNameAndSymbol

Issue_40	Mutable savings name invalidates and can Later revive EIP-712 authorizations
Severity	**Low**
Description	SavingsNameable.setNameAndSymbol() allows governance to change

	the Savings token's name after deployment.
	
	The name does not merely display metadata. EIP3009._domainSeparator() dynamically hashes the current result of name(). Consequently:
	
	• changing the name immediately invalidates every outstanding authorization signed under the old name; and
	• restoring the exact old name reconstructs the old domain separator, making still-valid and unused old signatures executable again.
	
	This affects standard EIP-3009 transfers and cancellations as well as Savings-specific deposit and redemption authorizations.
	
	Only the name matters. Changing the symbol while preserving the name does not alter the signing domain.
	
	Root Cause
	
	SavingsNameable returns the mutable __name:
	
	function name() public view returns (string memory) {
	  return __name;
	}
	
	Governance may overwrite it at any time:
	
	function setNameAndSymbol(
	  string memory newName,
	  string memory newSymbol
	) external restricted {
	  _setNameAndSymbol(newName, newSymbol);
	}
	
	Meanwhile, EIP3009 dynamically calculates:
	
	keccak256(
	  abi.encode(
	  EIP712_DOMAIN_TYPEHASH,

```
        keccak256(bytes(name())),
        VERSION_HASH,
        block.chainid,
        address(this)
    )
);
```

The nonce state is stored independently of the name. A failed execution under the new name does not consume the authorization.

The interaction is visible in SavingsNameable.sol and EIP3009.sol.

Preconditions

- A user has signed an authorization under name N1.
- The authorization remains unused and unexpired.
- Governance changes the Savings name to N2.
- Governance later restores the exact name N1.
- The chain ID, verifying contract, and EIP-712 version remain unchanged.

Failure Scenario

1. Alice signs a redemption authorization under the name Savings USDp.
2. Governance renames the token to Parallel Savings USDp.
3. Alice's signature stops validating because the domain separator has changed.
4. The authorization nonce remains unused.
5. Governance later restores the name Savings USDp.
6. The original domain separator is reconstructed.
7. A relayer holding Alice's old authorization can execute it while validBefore has not elapsed.

Impact

Outstanding integrations can fail unexpectedly following an ordinary branding change. Restoring a historical name can reactivate authorizations users or relayers may have treated as obsolete.

	This does not permit signature forgery: • the original amount, receiver, validity window, and other signed fields remain binding; • consumed or canceled nonces cannot be revived; and • expired authorizations remain expired.
Recommendations	Consider acknowledging this issue.
Comments / Resolution	Acknowledged.

BaseSavings

The BaseSavings contract is the abstract upgradeable foundation of the Parallel Savings vaults. It combines ERC-4626 accounting, access management, and UUPS upgradeability, while restricting implementation upgrades to authorized governance and defining the common environment in which deposited tokenP earns distributed yield.

Privileged Functions

- nond

EIP3009

The EIP3009 contract is an abstract token extension that enables transfers through signed EIP-712 authorizations without requiring a prior allowance transaction. It supports EOA and EIP-1271 signatures, validity windows, per-authorizer nonce consumption, authorization cancellation, unrestricted direct transfers, and payee-restricted `receiveWithAuthorization` calls for atomic integrations.

Privileged Functions

- none

Savings

The Savings contract is an upgradeable ERC-4626 vault through which users deposit tokenP and receive yield-bearing vault shares. It accrues governance-configured yield by minting additional tokenP, tracks backing through `storedAssets` to exclude direct donations, supports standard and EIP-3009-authorized deposits and redemptions, and provides controlled rate, pause, updater, and surplus-recovery functionality.

Privileged Functions

- pause
- unpause
- toggleTrusted
- setRate
- setMaxRate
- recoverSurplus

Issue_41	Permissionless zero-value checkpoints make savings yield and tokenP issuance interaction-frequency dependent
Severity	**Low**
Description	Every ordinary ERC-4626 interaction calls _accrue() before processing its amount. None rejects a zero amount.

Anyone can therefore checkpoint the Savings contract through:

deposit(0, receiver)
mint(0, receiver)
withdraw(0, receiver, owner)
redeem(0, receiver, owner)

The accrual calculation is a cubic truncation of compound growth rather than a path-independent index. In addition, each checkpoint floors the computed asset balance and unconditionally advances lastUpdate.

As a result, Savings yield and newly minted TokenP depend on how often users interact with the contract.

Root Cause

The growth factor implemented by _computeUpdatedAssets() is:

1
+ t·r
+ C(t,2)·r²
+ C(t,3)·r³

It omits the fourth and higher-order terms of (1 + r)^t.

Splitting an interval into multiple checkpoints multiplies several truncated factors together, introducing positive cross-terms that a single long checkpoint omits.

At the same time, every checkpoint floors:

currentBalance * growthFactor / BASE_27

and _accrue() advances lastUpdate even when earned == 0. Fractional earnings are not carried forward.

The behavior is present in Savings.sol.

Preconditions

- Savings is initialized and unpaused.

- storedAssets > 0.
- rate > 0.
- An attacker can repeatedly submit ordinary zero-value ERC-4626 calls.
- The rate, balance, and checkpoint frequency produce a difference that survives integer rounding.

Failure Scenario A: Yield Suppression Through Rounding

Assume the interest earned over one block is less than one smallest TokenP unit.

1. Without intervention, multiple blocks accumulate until at least one unit can be minted.
2. An attacker calls deposit(0, attacker) every block.
3. Each _computeUpdatedAssets() result rounds back to storedAssets.
4. _accrue() nevertheless sets lastUpdate to the current timestamp.
5. The fractional accrual interval is discarded.
6. Repetition can indefinitely delay the first unit of yield.

Failure Scenario B: Higher Issuance Through Frequent Compounding

Assume balances are large enough that rounding is negligible.

1. With no interaction over interval T, the vault applies one cubic factor F(T).
2. With checkpoints over T1 + T2 = T, it applies:

F(T1) × F(T2)

3. The multiplication introduces positive fourth- and higher-order cross-terms missing from F(T).
4. More frequent checkpointing therefore produces more TokenP than one long inactive interval.

The exact direction at small balances depends on whether the positive compounding difference or repeated flooring dominates. That uncertainty is itself the invariant violation: issuance is controlled by

	interaction cadence. Impact A public caller can alter: • the yield allocated to Savings holders; • the amount of TokenP minted into Savings; and • potentially the broader TokenP accounting mismatch caused by Savings issuance. At ordinary rates with 18-decimal assets, per-checkpoint rounding losses may be economically small. The discrepancy grows with longer inactive periods, higher rates, larger balances, or deliberately frequent checkpointing.
Recommendations	Since even adding a zero-amount revert, we do not recommend any change. This issue is inherent within the mathematical principle of contract.
Comments / Resolution	Acknowledged.

Issue_42	Savings issuance can corrupt the reported collateral ratio
Severity	**Low**
Description	The developer explains that Savings minted amount is only the maximum of the previously created headroom via surplus.

The intended sequence is:

1. Surplus.processSurplus() mints TokenP against collateral already held by the Parallelizer, increasing tracked issuance.
2. Surplus.release() burns the zero-address payee's TokenP without reducing tracked issuance.
3. This creates headroom between tracked issuance and outstanding TokenP.
4. Savings._accrue() later mints TokenP into that headroom.

Inside that bound, Savings minting does not create an unbacked liability. The problem is that the bound exists only in off-chain rate calculations.

Root Cause

Savings._accrue() executes:

ITokenP(asset()).mint(address(this), earned);
storedAssets = newTotalAssets;

It does not:

- read the available surplus-generated headroom;
- cap earned to that headroom;
- synchronize atomically with processSurplus() or release(); or
- stop accrual when cumulative Savings issuance has consumed the available headroom.

Meanwhile, LibGetters.getCollateralRatio() continues deriving recognized issuance solely from:

normalizedStables * normalizer / BASE_27

The intended invariant is therefore operational rather than enforced:

cumulative Savings issuance ≤ cumulative pre-funded TokenP burn headroom
Preconditions

- Savings has the TokenP minter permission.
- A positive rate remains active.

- Accrual exceeds the remaining surplus-generated headroom.
- This can happen through delayed harvesting, insufficient future surplus, an inaccurately calculated rate, or continuing accrual after the initially budgeted period.
- A Savings interaction triggers _accrue() and shareholders can subsequently withdraw the minted TokenP.

Failure Scenario

Assume:

Collateral value: $100
Parallelizer tracked issuance: 100 TokenP
Outstanding redeemable claims: 100 TokenP
Available headroom: 0 TokenP

Savings accrues and mints 5 TokenP:

Reported collateral ratio: 100 / 100 = 100%
Economic collateral ratio: 100 / 105 ≈ 95.24%

A first mover redeeming 5 TokenP is priced using tracked issuance of 100 rather than actual claims of 105. If the redemption curve pays approximately 99.5%, the first mover receives about $4.975 of collateral, whereas a genuinely proportional claim against all 105 TokenP would be approximately $4.762 before applying the curve.

The redemption then:

- burns 5 actual TokenP; and
- subtracts approximately 5 from tracked issuance.

The state becomes approximately:

Remaining claims: 100
Tracked issuance: 95
Persistent discrepancy: 5

The mismatch is transferred to later holders rather than reconciled.

Impact

	The Parallelizer overstates its collateral ratio and prices early exits as if excess Savings-minted claims did not exist. Early holders can withdraw more collateral than their economically proportional entitlement, socializing the shortfall to later holders. If the mismatch persists until tracked issuance approaches its terminal guards, a tail of TokenP can remain outstanding while ordinary burn and redemption routes refuse to consume the final tracked issuance.
Recommendations	Consider under all circumstances permanently evaluating the savings rate.
Comments / Resolution	Acknowledged.

SavingsEIP3009

The SavingsEIP3009 contract is an abstract authorization extension that defines Savings-specific signed deposit and redemption intents. It binds each authorization to the vault, owner, receiver, asset or share amount, validity window, and nonce, while sharing the replay-protection state of the underlying EIP-3009 implementation.

Privileged Functions

- none

AccessManaged

The AccessManaged contract provides non-upgradeable contracts with function-level permissions controlled by an external OpenZeppelin-compatible access manager. Its `restricted` modifier supports both immediate permissions and delayed scheduled operations, while authority replacement can only be initiated by the currently configured authority.

Privileged Functions

- setAuthority

AccessManagedUpgradeable

The AccessManagedUpgradeable contract provides the upgrade-safe equivalent of the protocol's access-management framework. It stores authority and scheduled-operation state in an ERC-7201 storage namespace, supports initializer-based configuration, and enforces immediate or delayed function permissions through the connected access manager.

Privileged Functions

- setAuthority

Token

FlashParallelToken

Privileged Functions

- setFlashLoanParameters
- setFlashLoanRecipient
- toggleActiveToken

BridgeableTokenP

The BridgeableTokenP contract is a LayerZero OFT bridge adapter for transferring Parallel tokens across supported chains. It burns either principal tokenP or its OFT representation on the source chain, credits principal tokenP within destination-specific daily and global limits, issues OFT tokens for any uncredited remainder, applies configured fees to eligible principal credits, and tracks cross-chain credit and debit exposure with optional isolation-mode restrictions.

Privileged Functions

- emergencyRescue
- toggleIsolateMode
- setFeesRate
- setDailyCreditLimit
- setGlobalCreditLimit
- setDailyDebitLimit
- setGlobalDebitLimit
- setFeesRecipient
- pause
- unpause

Issue_43	Routing principal through a credit-saturated chain bypasses the destination protocol fee
Severity	**Medium**
Description	Bridge fees are charged only when: 1. the destination actually credits principal TokenP; and 2. the message's feeApplicable flag is true. When a destination has no principal credit capacity, it mints lz-TokenP

instead and charges no fee.

If the user then bridges that lz-TokenP to another chain, the new source marks the message as fee-inapplicable. The final destination can convert the value into principal TokenP without charging its protocol percentage fee.

Root Cause

The fee flag represents only the asset burned for the most recent bridge hop:

principal TokenP burned → feeApplicable = true
lz-TokenP burned → feeApplicable = false

It does not preserve the value's original principal-token provenance across intermediate hops.

Additionally, destination fees are charged only inside:

```
if (amountReceived > 0) {
  if (_isFeeApplicable) {
    feeAmount = amountReceived.percentMul(feesRate);
  }
}
```

If principal credit capacity is zero:

amountReceived = 0

No fee is charged, and _credit() mints the remainder as lz-TokenP.

Preconditions

- Chain A can debit principal TokenP.
- Intermediate chain B has exhausted principal credit capacity.
- Chain C has available principal credit capacity.
- A→B and B→C LayerZero peers are configured.
- The user can afford two LayerZero messages.
- Relevant debit and credit limits permit the route.

	Failure Scenario Assume: Amount: 1,000 USDp Chain C fee: 1% Chain B capacity: 0 Chain C capacity: 1,000+ Direct A → C A burns principal TokenP, so: feeApplicable = true C credits principal: User receives: 990 USDp Fee recipient: 10 USDp A → saturated B A again burns principal TokenP: feeApplicable = true B has no principal capacity: Principal credited: 0 Fee charged: 0 lz-TokenP minted: 1,000 B → C The user burns lz-TokenP on B, so the new message carries: feeApplicable = false C has principal capacity and credits the entire amount: User receives: 1,000 USDp Fee recipient: 0 USDp Final Comparison Direct A → C:

	User receives 990 USDp Protocol receives 10 USDp A → saturated B → C: User receives 1,000 USDp Protocol receives 0 USDp The user incurs an additional LayerZero execution cost but avoids the percentage fee. Impact • Protocol bridge revenue can be bypassed. • Users are economically incentivized to route through saturated deployments. • Larger transfers make the fixed extra messaging cost negligible relative to the percentage fee. • The bypass increases the profitability of cross-chain negative-fee and oracle-state arbitrage. • Credit saturation changes from a throughput safeguard into a fee-avoidance primitive. **Likewise, general OFT bridging requires no fee at all.**
Recommendations	Since the protocol currently enforces no fees and this issue has been acknowledged in previous reports, no change is required. To fix this issue in its root, the fee mechanism needs to be refactored to apply a fee on all cross chain transfers.
Comments / Resolution	Acknowledged.

Issue_44	Mutable feesRate can reprice pending Lz-Token conversions and in-flight transfers
Severity	**Medium**
Description	Users cannot commit to a minimum amount of principal TokenP when converting lz-TokenP. swapLzTokenToPrincipalToken() calculates the fee using the live feesRate and accepts no minPrincipalOut or maxFeeRate. An owner fee update can therefore execute before a pending conversion and cause the user to receive less than expected. The same problem affects cross-chain messages: the source message

commits only to whether a fee is applicable. The destination calculates the actual fee using its current feesRate when _lzReceive() executes.

Root Cause

The local conversion burns the creditable lz-token amount and then reads mutable storage:

```
uint256 feeAmount =
   totalPrincipalTokenAmountToCredit.percentMul(feesRate);

uint256 principalTokenAmountCredited =
   totalPrincipalTokenAmountToCredit - feeAmount;
```

There is no user-supplied lower bound for principalTokenAmountCredited.

For bridged principal, the message includes feeApplicable, but neither the fee rate nor a minimum net-principal amount. _handleCreditPrincipalToken() applies the destination's current rate:

```
if (_isFeeApplicable && feesRate > 0) {
   feeAmount = amountReceived.percentMul(feesRate);
   amountReceived -= feeAmount;
}
```

The relevant flows are in BridgeableTokenP.sol.

Preconditions

For the local route:

- principal credit capacity is available;
- the user submits swapLzTokenToPrincipalToken();
- the owner increases feesRate before execution.

For the cross-chain route:

- principal TokenP was sent with feeApplicable = true;
- the destination has credit capacity;
- feesRate increases between source debit and destination

	delivery. Failure Scenario Assume the current fee is 0%. 1. A user submits a conversion of 1,000 lz-USDp. 2. Governance raises feesRate to 10%. 3. The fee update executes first. 4. The user's transaction still succeeds. 5. It burns 1,000 lz-USDp but returns: 900 principal USDp to the user 100 principal USDp to the fee recipient The function's return value is not protection because the transaction cannot express a condition against it. Impact Pending users and in-flight bridge messages can receive up to the configured maximum fee less than the amount anticipated when submitting the operation. No public attacker can independently change the fee, so this is principally a governance/configuration race rather than an unprivileged theft vector.
Recommendations	Consider communicating a fee change upfront with the community.
Comments / Resolution	Acknowledged.

Issue_45	Cross-Chain fungibility exposes independent Parallelizer states to economic arbitrage
Severity	**Low**
Description	BridgeableTokenP permits users to mint TokenP against collateral under one chain's Parallelizer state and burn the bridged TokenP against reserves governed by another chain's state. The bridge does not inspect: • source or destination collateral ratios; • Parallelizer oracle prices; • mint or burn fee curves; • paused collateral states; • collateral composition; or • economic differences between chains. It only enforces bridge credit/debit limits and, where applicable, its

	percentage fee. Consequently, any favorable mint condition on chain A can be combined with any favorable burn or redemption condition on chain C. Root Cause When principal TokenP is bridged: • the source BridgeableTokenP burns principal TokenP; • the destination BridgeableTokenP mints principal TokenP; • no Parallelizer state is consulted. The destination TokenP is fully fungible and accepted by its local Swapper and Redeemer. The bridge's creditDebitBalance, daily limits and global limits constrain quantity, but not economic value or reserve quality. Failure Scenario Assume: Chain A: Minting 100 TokenP costs $98 of collateral due to local oracle/deviation/fee state. Chain C: Burning 100 TokenP returns approximately $100 of healthy collateral. A user can: 1. Mint 100 TokenP on A for $98. 2. Bridge the principal or convert through lz-TokenP. 3. Receive principal TokenP on C. 4. Burn it against C's healthy reserves for approximately $100. The spread is profitable if it exceeds: • bridge fees;

	• local mint and burn fees; • LayerZero execution costs; and • transaction costs. The also identified fee-free lz-TokenP route can reduce one of these barriers. Accounting Effect The sequence reallocates reserve backing and attributed issuance: • A gains collateral and increases locally attributed issuance. • C loses collateral and decreases locally attributed issuance. This can preserve each chain's local accounting ratio when prices are equal, meaning the absence of normalizer changes during the bridge itself may be intentional. The economic issue is that different local pricing and risk states determine the value transferred between pools. Impact • Healthy reserves on one chain can absorb losses or favorable pricing from another chain. • Local mint caps on C do not prevent imported TokenP from consuming C's reserves. • A depeg or permissive fee curve on one chain can contaminate stronger chains. • Bridge credit limits bound throughput but do not prevent profitable execution. • Message latency gives users time to select destination execution based on changing chain states.
Recommendations	This issue is design inherent. There is no simple fix rather than keeping reasonable high-fees.
Comments / Resolution	Acknowledged.

Issue_46	Permissionless tokenP burns leave phantom issuance in parallelizer accounting
Severity	**Low**
Description	Any TokenP holder can call burnStablecoin() and destroy their TokenP directly. This reduces TokenP.totalSupply() without updating the Parallelizer's global normalizedStables, per-collateral normalizedStables or normalizer. The Parallelizer therefore continues treating the burned TokenP as outstanding issuance even though the corresponding tokens no longer exist. The collateral claim represented by the burned TokenP is not redistributed to remaining holders or recognized as surplus. Root Cause TokenP.burnStablecoin() is permissionless and performs only an ERC-20

supply burn:

```
function burnStablecoin(uint256 amount) external {
  _burn(msg.sender, amount);
}
```

Unlike Parallelizer burn and redemption flows, it does not execute through a function that updates Parallelizer issuance accounting.

The Parallelizer calculates total issued stablecoins from its independent accounting:

```
uint256(ts.normalizedStables)
  * uint256(ts.normalizer)
  / BASE_27
```

Similarly, the denominator used by LibGetters.getCollateralRatio() is derived from:

```
stablecoinsIssued =
  uint256(ts.normalizedStables).mulDiv(
    ts.normalizer,
    BASE_27,
    Math.Rounding.Ceil
  );
```

Neither calculation reconciles against TokenP.totalSupply().

Because TokenP and the Parallelizer are separate contracts, the direct burn cannot update the Diamond's storage.

Preconditions

- The caller owns TokenP and calls burnStablecoin().
- The burned TokenP represents issuance tracked by the Parallelizer.
- No subsequent governance accounting adjustment compensates for the burn.
- The burn is not offset against another intended source of

untracked TokenP issuance.

Failure Scenario

Assume a simplified state in which:

Actual TokenP supply: 1,000
Parallelizer tracked issuance: 1,000
Collateral value: 1,100 USD

A holder calls:

burnStablecoin(100e18);

The resulting state is:

Actual TokenP supply: 900
Parallelizer tracked issuance: 1,000
Collateral value: 1,100 USD

The actual collateralization relative to circulating TokenP is approximately:

1,100 / 900 = 122.22%

The Parallelizer continues reporting:

1,100 / 1,000 = 110%
The nonexistent 100 TokenP remains represented in the Parallelizer's redemption and collateral accounting.
Impact
Direct burns create a persistent divergence between actual supply and Parallelizer-tracked issuance.

Consequences include:

- the protocol understates collateralization relative to actual outstanding TokenP;
- the burned holder's collateral claim is not redistributed to remaining holders;
- collateral associated with the phantom issuance is not

	automatically recognized as surplus; • redemption calculations continue reserving value for TokenP that no longer exists; • repeated burns can leave an increasing amount of collateral economically stranded. The direct path requires the holder to destroy their own TokenP and therefore does not provide an obvious profitable attack. Its practical severity is consequently limited and may be classified as self-harm or intended settlement.The issue becomes more relevant if an external settlement mechanism relies on burnStablecoin() as an economically meaningful settlement primitive, because that integration may incorrectly assume the burn reduces backing liabilities.
Recommendations	Consider removing the capability to burn tokens, users can simply transfer it to 0xdead.
Comments / Resolution	Acknowledged.